



Runi

Run with Intelligence

תיק הגשה סופי – פרויקט גמר

מאמן ריצה אישי מבוסס AI: תוכניות אימון מותאמות, מעקב ותובנות לרצים ולמאמנים.
אפיון מוצר • ארכיטקטורה • תכנון טכני • בדיקות • אבטחת מידע • סקייל וביצועים

קורס: טכנולוגיות אינטרנט ופיתוח פול-סטאק

מרצה: מר אוהד אסולין

מגיש: שמואל אסמרה

תוכן עניינים

מספרי העמודים הם מיקום בקובץ המאוחד; לכל מסמך תוכן עניינים פנימי ומספור עמודים משלו, והמסמכים מצורפים גם כקבצים נפרדים.

4 מסמך אפיון בדיקות 35

- 36 מה נבדק, ולמה דווקא כך
- 37 1. בדיקות לפיצ'רים המרכזיים
- 39 2. בדיקות לקלטים לא תקינים
- 40 3. בדיקות לתהליכים עסקיים
- 40 4. בדיקות הרשאות
- 41 5. בדיקות למסד הנתונים
- 41 6. בדיקות למקרי קצה
- 42 7. בדיקות UI
- 42 8. בדיקות ידניות מתועדות
- 43 9. מגבלות ומה היה נבדק בהמשך

5 מסמך אבטחת מידע 44

- 45 הרעיון המרכזי
- 45 1. אימות (Authentication)
- 46 2. הרשאה (Authorization)
- 47 3. פעולות שדורשות משתמש מחובר
- 47 4. מניעת גישה לנתוני משתמש אחר
- 49 5. ולידציה של קלטים
- 49 6. הגנה על קריאות ל-API
- 50 7. שמירת סודות
- 51 8. מה עדיין פתוח

6 מסמך סקייל וביצועים 53

- 54 נקודת המוצא
- 54 1. מה יקרה עם עשרות או מאות משתמשים
- 55 2. אילו שאילתות עלולות להיות כבדות
- 56 3. אינדקסים
- 57 4. איך נמנעים מטעינה מיותרת
- 57 5. Pagination — ואיפה הוא חסר
- 58 6. הפרדה בין צד לקוח לצד שרת
- 59 7. מגבלות המוצר הנוכחי
- 60 8. מה היינו משפרים לגרסה הבאה

1 מסמך אפיון מוצר 3

- 4 1. הבעיה
- 4 2. המשתמשים
- 4 3. הלקוחות והמודל העסקי
- 5 4. מטרות המוצר
- 6 5. יכולות התוכנה שנבנו
- 7 6. חוויית משתמש ותצוגת נתונים
- 8 7. התהליכים המרכזיים
- 9 8. מקורות נתונים ואינטגרציות
- 9 9. מחוץ ל-Scope הנוכחי
- 10 נספח — מצב המוצר נכון להיום

2 מסמך ארכיטקטורת תוכנה 11

- 12 1. עקרונות ארכ' מוצר
- 12 2. ישויות וטבלאות — הישויות מקובצות לפי שלושה נושאים
- 13 3. עמודי האפליקציה
- 15 4. נקודות הקצה
- 16 5. זרימת המידע
- 18 6. משתמשים והרשאות
- 20 7. ספריות ושירותים חיצוניים

3 מסמך תכנון טכני מפורט 23

- 24 עקרונות התכנון
- 24 1. מבנה התיקיות
- 25 2. הקומפוננטות המרכזיות
- 26 3. מבנה בסיס הנתונים
- 28 4. פעולות CRUD מרכזיות
- 29 5. תיאור ה-API
- 30 6. הלוגיקה העסקית
- 32 7. ניהול State
- 32 8. טיפול בשגיאות
- 33 9. ולידציות של קלטים
- 34 10. חוויית המשתמש המרכזית

מסמך 1 – אפיון מוצר

תוכן עניינים

1.	הבעיה	2
2.	המשתמשים	2
3.	הלקוחות והמודל העסקי	2
4.	מטרות המוצר	3
5.	יכולות התוכנה שנבנו	4
6.	חויית משתמש ותצוגת נתונים	5
	כלל שנאסף לאורך כל המוצר	5
	הגרפים	5
7.	התהליכים המרכזיים	6
	למה ההצטרפות היא בקוד ולא בהזמנה במייל	6
8.	מקורות נתונים ואינטגרציות	7
9.	מחוץ ל-Scope הנוכחי	7
	נספח – מצב המוצר נכון להיום	8

1. הבעיה

למי	הבעיה	הפתרון
רץ	מתאמן לפי תוכנית שלא מכירה אותו – קובץ PDF או תבנית אחידה שלא יודעת כמה ישן, מה הדופק אמר אתמול, או שהנפח קפץ מהר מדי. החלטות (להעמיס / להקל / לנוח) נלקחות בתחושה, בלי גיבוי נתונים – התוצאה השכיחה: פציעת עומס-יתר או הגעה עייפה למרוץ.	תוכנית שנגזרת מהפרמטרים של הרץ עצמו, מתעדכנת אחרי כל ריצה, ומסבירה כל מספר שהיא מציגה – בממשק נקי.
מאמן	מלווה כמה מתאמנים בלי מקום אחד שמתכלל את המוכנות של כולם ואומר לו בבוקר למי לשים לב – פותח כל מתאמן בנפרד, או לא פותח בכלל. ככל שהרוסטר גדול יותר, קשה יותר לרדת לרמת הדיוק והניואנס האישי.	לוח בקרה שמתרגם את נתוני כל המתאמנים לטריאז' מיידי של הרוסטר – מי צריך תשומת לב עכשיו, בלי לפתוח כל פרופיל בנפרד.

מה שקיים היום – Runalyze ו-Strava, Garmin, TrainingPeaks, intervals.icu – עוסק בעיקר בניתוח אתלטים מקצועיים ובהצגת נתונים לאחר; הכלים חזקים אך צפופים, מורכבים, ומשאירים את הפרשנות למשתמש. Runi שואפת להיות פלטפורמה שעוזרת ללמוד מהנתונים ולהתקדם, ועושה התאמות קדימה לתוכנית האימונים – בין אם יש לך ליווי מקצועי ובין אם אתה מתאמן לבד.

2. המשתמשים

סוג משתמש	הצורך
רץ (מתחיל / מנוסה)	בין אם זה רץ מתחיל שמתכוון למרוץ ראשון או רץ מנוסה שרוצה תוכנית שמגיבה לביצועים בפועל – המטרה זהה: תוכנית הדרגתית שלומדת את הנתונים שלך ועוזרת לך לבצע התאמות.
מאמן	מלווה כמה מתאמנים; צריך יכולת לרדת ממבט-על על כל הרוסטר עד לרמת הדופק באימון בודד. מאמן הוא לרוב גם רץ בעצמו – כך שחשבון אחד יכול לשמש את שני התפקידים.

3. הלקוחות והמודל העסקי

שני לקוחות. הרץ מקבל את הפלטפורמה בחינם; המאמן משלם. לשניהם שתי חבילות, ולרץ החבילה הבסיסית היא כל המוצר.

הרץ הוא לקוח לכל דבר – המוצר שלם גם בלי מאמן – והבסיס שלו חינמי וכולל את כל היכולות: תוכנית, מוכנות, לוח המספרים, Ask Runi. חבילת פרימיום לרץ (\$10 לחודש) תיפתח בעתיד עם RunAI – שיחה חופשית עם Runi על האימונים שלו – ומוצגת כבר היום בהגדרות כ"בקרוב", בלי כפתור רכישה. הוא מביא את הנתונים ואת השימוש היומיומי, והוא מה שמביא מאמנים: מאמן מצטרף למקום שבו הרצים שלו כבר נמצאים.

המאמן הוא מקור ההכנסה. הוא מרוויח מהליווי, והכלי חוסך לו זמן ומקטין את הסיכון שמתאמן שלו יפצע – ולכן יש לו נכונות תשלום. המנוי נבחר בהגדרות המאמן, בשתי חבילות:

פרימיום	בסיס	
10\$ לחודש	חינם לחצי שנה, ואז 5\$ לחודש	מחיר
ללא הגבלה	עד חמישה	מתאמים
כל יכולות הבסיס, ובעתיד גם צ'אט חופשי עם Runi דרך מודל שפה	רוסטר, מחזורים, תבניות, עריכת אימונים	יכולות

הבחירה בין החבילות נעשית בהגדרות המאמן, בלשונית החיוב: שני כרטיסי החבילות זה לצד זה, החבילה הנוכחית מסומנת, ומתחתם טופס אמצעי תשלום – מוקאפ המסומן ככזה, שאינו שומר ואינו מחייב. למאמן יש דף הגדרות אחד, גם כשהוא במצב "האימון שלי", כך שחבילת רץ אינה יכולה להגיע אליו בטעות.

4. מטרות המוצר

למי	מטרה	איך נמדוד
אתלט	להגיע ליום המרוץ מוכן ובלי פציעה	אחוז מתאמים שסיימו תוכנית בלי הפסקה מפציעה
	דבקות בתוכנית	אחוז אימונים שבוצעו מתוך המתוכננים
	להבין למה, לא רק מה	פתיחת פירוק ציון המוכנות ולוח "המספרים שלך"; ירידה בפניות למאמן
מאמן	לצמצם זמן טריאז'	זמן עד זיהוי מתאמן בסיכון
	הכנסה חוזרת	שיעור המאמנים שעוברים לפרימיום; הכנסה חודשית למאמן
שניהם	בסיס משתמשים בחיכוך כניסה נמוך	הרשמה עד סנכרון ראשון

5. יכולות התוכנה שנבנו

יכולת	פירוט
ניהול משתמשים	הרשמה והתחברות עם בחירת תפקיד – אתלט או מאמן (Supabase Auth) - אימות כתובת במייל, שחזור סיסמה, התנתקות - קוד הצטרפות: מאמן יוצר קוד, מתאמן מקליד אותו ויוצר את הקשר
חיבור וניתוח נתונים	חיבור שיעון (Garmin / Polar / Coros / Suunto / Strava) דרך intervals.icu כשכבת-ביניים - לכל ריצה: מקטעים גולמיים (דופק, קצב, גובה, קדנס) וניתוח נגזר – צורת קצב, שיאים אישיים, סחיפת דופק - מודל עומס ומוכנות יומי – כושר / עייפות / טריות (CTL / ATL / TSB), יחס עומס (ACWR), ציון מוכנות 0-100 (ראה נספח)
תוכנית אימונים ומעקב	שלושה נתיבים לתוכנית: קוד ממאמן, תוכנית Runi עם תצוגה מקדימה, או תוכנית משלו - התאמה אוטומטית מדי לילה לפי המוכנות, עם הסבר – וחזרה לאחר כשהסיבה חולפת - ספירה לאחר ותחזית זמן סיום למרוץ
כלי המאמן	רוסטר עם דגלי תשומת לב לפי ספים שהמאמן קובע - מחזורי הכנה מתבניות: שיבוץ רצים, עריכת שם / תאריך / תבנית, סגירה - עריכת אימון של מתאמן – והמנוע לא דורס אותה
חבילות וחיוב	שתי חבילות לרץ ושתיים למאמן, נבחרות בלשונית החיוב בהגדרות - מכסת המתאמנים של חבילת הבסיס נאכפת במסד, בפונקציית ההצטרפות - טופס אמצעי תשלום כמוקאפ מסומן – הסליקה עצמה מחוץ ל-Scope
שמירת נתונים	מסד Supabase עם הרשאות ברמת שורה (RLS) – כל משתמש רואה רק את הנתונים שלו - כל מספר על המסך ניתן לעקוב עד לחישוב שהפיק אותו – שקיפות מלאה
חווית משתמש	פאנל Ask Runi – שאלות על הנתונים מכפתור צף בכל מסך - הסבר לכל ציון / מדד – מה נמדד, כמה זה שוקל, ומה זה אומר

6. חוויית משתמש ותצוגת נתונים

מהות המוצר בחוויית המשתמש היא הנגשת המידע: ממשק אינטואיטיבי שיוצר למתאמן מרחב אישי בפלטפורמה, בנוי כולו על הנתונים שלו – עם הסבר לכל נתון, והצגה ברורה של כל מדד ואופן החישוב שלו.

עיקרון	אצל המתאמן	אצל המאמן
עיצוב	בהירות לפני צפיפות – כל מסך עונה על שאלה אחת (כמה אני מוכן? מה מחר? איך היה האימון?), במערכת עיצוב אחת ועקבית: לכל מדד צבע וסמל קבועים, זהים בכל מסך.	אותה שפה עיצובית בדיוק ברוסטר וביומן – אין 'מצב מאמן' נפרד, כך שמאמן שהוא גם רץ לא לומד שתי שפות.
מעבר ממאקרו למיקרו	ירידה לרזולוציה בלחיצה אחת – מבט-על מהיר על התוכנית, ולחיצה חושפת את הפרטים עד גרף הדופק של הקילומטר הבודד.	אותו עיקרון על כל הרוסטר: מבט-על על כל המתאמנים, ולחיצה יורדת לרמת האימון הבודד של מתאמן ספציפי.
ניתוח נתונים	גרפים לכל מדד (מוכנות, כושר/עייפות/טריות, נפח, קצב, דופק) וגרפי שיפור לאורך זמן. אמינות: בלי נתון אמיתי – אין מדד.	אותם עקרונות ברמת הרוסטר: גרף מגמה לכל מתאמן – בלי נתון בדוי גם כאן.
רגעי שיא והצלחה	בשגרה: שיא אישי, רצף ימים, סגירת שבוע. לקראת מרוץ: ספירה לאחור עם תחזית.	יומן צבוע לפי מרוץ יעד ורוסטר עם דגלי תשומת לב – כדי לראות מי מתקרב לרגע שלו.
תכנון מול ביצוע	כל אימון מציג תוכן מול בפועל, עם פסיקה מבוססת קצב (לא רק מרחק) – ורק כשיש אימון מתוכנן אמיתי להשוואה.	אותה השוואה ברמת המאקרו: תצוגת Planned / Ran / Gap / על פני כל הרוסטר והמחזור, בלי לפתוח כל מתאמן בנפרד.

כלל שנאכף לאורך כל המוצר

בלי נתון אמיתי – לא מציגים מספר. קו מקף, מצב ריק, או הסתרה. לא ערך לדוגמה ולא ברירת מחדל שנראית כמו מדידה.

הכלל לא היה מובן מאליו – סריקה שיטתית מצאה ותיקנה 47 מספרים שהוצגו כאילו היו אמיתיים (ראו מסמך אפיון בדיקות, סעיף 8).

הגרפים

באימון הבודד: קצב לאורך המרחק • דופק לאורך זמן עם סימון היכן הסחיפה התחילה • חלוקה לאזורי דופק • והשוואה למתוכנן.

בדשבורד: ציון מוכנות • כושר / עייפות / טריות לאורך הזמן • נפח שבועי • לוח שנה של הפעילות • ספירה לאחור עם תחזית • שיאים אישיים.

בלוח המספרים: גרף היסטוריה לכל מדד, עם טווחי הקריאה כרקע, בטווחי שבוע / חודש / שלושה חודשים / שנה.

אצל המאמן: יומן צבוע לפי מרוץ יעד • רוסטר עם דגלי תשומת לב • גרף מגמה לכל מתאמן.

כל הגרפים כתובים ידנית ב-SVG, בלי ספריית גרפים. ההחלטה הזו נלקחה כדי לשלוט לגמרי במראה, ולא לשלוח לדפדפן חבילה גדולה עבור קומץ גרפים.

7. התהליכים המרכזיים

מה המוצר מאפשר	תהליך
הרשמה עם בחירת תפקיד, אימות במייל, התחברות, שחזור סיסמה, התנתקות.	הרשמה והתחברות
חיבור השעון בהגדרות - סנכרון – ריצות ונתוני התאוששות נכנסים, והמקטעים נותחים - בחירת נתיב תוכנית – קוד מאמן, תוכנית של Runi עם תצוגה מקדימה, או תוכנית משלו - מעקב יומיומי – הבית מציג מוכנות, את האימון הבא, את הספירה לאחור ואת ההסבר - התאמה אוטומטית מדי לילה, עם הסבר וחזרה לאחור כשהסיבה חולפת - עזיבת תוכנית והתחלה של אחרת, מתי שרוצים	אתלט
יצירת קוד הצטרפות - המתאמן מקליד את הקוד בהגדרות שלו – וכך נוצר הקשר - המאמן פותח מחזור למרוץ ומשבץ אליו רצים – כל רץ מקבל תוכנית מהתבנית של המחזור, בשבוע שלו - הרוסטר, היומן והמחזורים מתמלאים - כניסה למתאמן, עריכת אימון – והמנוע לא ידרוס אותה - תזכורות פרטיות והעדפות - בחירת חבילה בלשונית החיוב בהגדרות, ומתחתיה טופס אמצעי תשלום (מוקאפ)	מאמן

למה ההצטרפות היא בקוד ולא בהזמנה במייל

זו החלטת מוצר, לא קיצור דרך. **הזמנה במייל מאפשרת למאמן ליצור קשר על מישהו בלי הסכמתו** – לכתוב כתובת ולקבל גישה לנתונים שלו.

הקוד הופך את הכיוון: המאמן מייצר קוד, נותן אותו למי שהוא רוצה, **והמתאמן הוא זה שיוצר את הקשר**. הקלדת הקוד היא ההסכמה.

זה גם נאכף במסד עצמו: מדיניות ההרשאה מאפשרת ליצור קשר מאמן-מתאמן רק למי שהוא המתאמן. ראו מסמך האבטחה – שם מתועד מה קרה כשזה לא היה כך.

בפועל זה כבר תהליך דו-כיווני: המאמן הוא זה שיוצר ומשתף את הקוד (היזמה שלו), והמתאמן משתמש בו כדי להשלים את הבקשה – רק שבשכבת המסד נרשמת רק פעולת הבקשה מצד המתאמן.

8. מקורות נתונים ואינטגרציות

תחום	מה קורה היום	לאן זה מכוון
מקורות נתונים ואינטגרציות	כל הריצות והמדדים הפיזיולוגיים מגיעים דרך intervals.icu כשכבת-ביניים, שכבר מסונכרנת מאחורי הקלעים עם רוב שעוני הריצה בשוק (Garmin, Polar, Coros, Suunto ויו) - כל מקור נכנס דרך שכבת ספק אחידה	חיבור ישיר לחלק מהיצרנים – בתהליך - מקור חדש = מיפוי נוסף על השכבה הקיימת, לא שינוי במוצר (ראה מסמך ארכיטקטורה)
שכבת האינטליגנציה	שלושה מקורות, ואף אחד מהם לא מודל שפה: תבניות (מדע אימון מקודד), נתונים (TRIMP, CTL/ATL/TSB, ACWR, ציון מוכנות) ושאלתה (Ask Runi) מרכיב תשובה מערכים שכבר חושבו). ההסבר מחושב, לא נוצר – ומכאן השם Run with Intelligence. פירוט: מסמך תכנון טכני, סעיף 6.	מודל שפה לצ'אט חופשי (RunAI), בחבילת הפרימיום – למתאמן ולמאמן: המודל מנסח, הפונקציות הקיימות מחשבות. Ask Runi של היום הוא השלד.

9. מחוץ ל-Scope הנוכחי

מה	למה לא, ומה היה נדרש
סליקת תשלומים אמיתית	Stripe Checkout מעל לשונית החיוב – טופס אמצעי התשלום כבר במקומו כמוקאפ. דורש חשבון עסקי; השלב הבא אחרי ההגשה
חיבור ישיר Garmin • Apple Health • Polar • Coros	בתהליך – דורש אישורי שותפות מהיצרנים. עד אז דרך המצבר; שכבת הספק כבר בנויה לקלוט אותם.
התראות Push	דורש אפליקציה או Web Push; אין הצדקה לפני שיש משתמשים
צ'אט חופשי עם Runi דרך מודל שפה	הגרסה הבאה, בחבילת הפרימיום. המודל מבין ומנסח; הפונקציות שנבדקו מחשבות
חשיפת שרת MCP לסוכני AI חיצוניים	מסלול הרחבה עתידי

נספח – מצב המוצר נכון להיום

- באוויר: <https://runi-coach.vercel.app>
- 835 בדיקות אוטומטיות ב-57 קבצים
- 24 מיגרציות מסד נתונים, 17 טבלאות מאחורי Row Level Security
- סריקת עומק מקצה לקצה שמצאה ותיקנה 47 ליקויים, כולל פרצת הרשאות
- ביקורת חישוב ממוקדת שסרקה את הקוד אחרי מחלקה אחת של טעות – ממוצע של ממוצעים, קצב שממוצע כאילו אינו הופכי, תאריכים בחשבון מילישניות – ומצאה ותיקנה תשע
- סבב מוצר אחרון (31 באוגוסט) – לוח המספרים עם היסטוריה, שלושת נתיבי התוכנית, מחזורי ההכנה של המאמן, דפי שגיאה ו-404 בעיצוב המוצר, ורישום שגיאות בצד השרת; נבדק בפרודקשן כמאמן וכרץ

מסמך 2 – ארכיטקטורת תוכנה

תוכן עניינים

1.	עקרונות ארכ' מוצר	2
2.	רכיבי המערכת	2
2.	ישויות וטבלאות- הישויות מקובצות לפי שלושה נושאים	3
3.	עקרונות עבודה בטבלאות	3
3.	עמודי האפליקציה	5
5.	אתלט	5
5.	מאמן	5
4.	נקודות הקצה	6
6.	Route Handlers – רק למה שמגיע מבחוץ	6
6.	Server Actions – כל השאר	6
5.	זרימת המידע	8
8.	מסך שנטען	8
8.	נתוני אימון נכנסים	8
9.	איך כל פעולת-כתיבה עובדת	9
6.	משתמשים והרשאות	10
10.	שני תפקידים, וחשבון אחד יכול להיות שניהם	10
10.	ההרשאה נאכפת במסד – כיסוי מלא	10
11.	פוסטגרס לא צועק כשהוא מסרב	11
7.	ספריות ושירותים חיצוניים	11
12.	ומה לא נכנס, בכוונה	12

1. עקרונות ארכ' מוצר

- **אין שרת נפרד** – Next.js מריץ קוד שרת בתוך אותו פרויקט, דרך Server Components ו-Server Actions. אין API נפרד לתחזק, אין שני מודלי טיפוסים שצריך לסנכרן, ואין CORS.
- **ההרשאות במסד, לא בקוד** – כל כלל הרשאה מוצמד לטבלה ונאכף בכל שאילתה על ידי Postgres עצמו, לא על ידי קוד האפליקציה. ראו סעיף 6.
- **מסד יחסי, לא מסמכי** – הנתונים כאן יחסיים מטבעם: תוכנית שייכת למרוץ, אימון שייך לתוכנית, מתאמן שייך למאמן. כל השאלות שהמוצר שואל הן צירופים.
- **Supabase בזכות RLS** – היכולת להצמיד לכל טבלה כלל שאומר מי רשאי לראות ולשנות כל שורה, והכלל נאכף במסד עצמו – לא באפליקציה. באג בקוד לא יכול לעקוף אותו.
- **פריסת קוד ומיגרציות הן שני דברים נפרדים** – Vercel פורס קוד ולא נוגע במבנה המסד, וגם לא יכול. הסדר קבוע: מבנה קודם, קוד אחריו – ותמיד יש חלון שבו המסד כבר חדש והקוד עוד ישן, ולכן מיגרציה חייבת להיות תואמת-לאחור.

רכיבי המערכת

ששה רכיבים, ואף אחד מהם לא שרת שאנחנו מתחזקים.

רכיב	מה הוא עושה	איפה רץ
אפליקציית Next.js	הממשק וגם הלוגיקה בצד השרת – אותו פרויקט	Vercel
Supabase Postgres	הנתונים, וגם אכיפת ההרשאות	Supabase
Supabase Auth	הרשמה, התחברות, אימות מייל, שחזור סיסמה	Supabase
משימה יומית	סנכרון, חישוב מוכנות והתאמת תוכנית ב-03:00	Vercel Cron
intervals.icu	מקור הנתונים – ריצות, מקטעים, שינה ו-HRV	חיצוני
Resend	שליחת מיילי Supabase Auth – אימות מייל ושחזור סיסמה	חיצוני

אימות ושגיאות – אימות: /login, /signup, /auth/callback, /auth/confirm, /auth/reset – ההתנהגות המדויקת מתועדת במסמך אבטחת המידע. שגיאות: error.tsx ו-not-found.tsx תופסים שגיאה לא צפויה ו-404 בהתאמה; global-error.tsx (תפיסת קריסה בתוך ה-root layout עצמו) חסר – זו נקודה פתוחה. שגיאות בפעולות שרת נרשמות בנפרד ביומן Vercel; כיסוי מקרי השגיאה בבדיקות מתועד במסמך אפיון הבדיקות.

2. ישויות וטבלאות- הישויות מקובצות לפי שלושה נושאים

פעילות ונתונים – מה שנכנס מהשעון: ריצות, שינה, דופק ו-HRV, ותמונת המוכנות היומית.

תוכנית – מה מתוכנן: מרוץ היעד, תוכנית האימונים עצמה, והשלד שממנו היא נבנית.

מאמן – ניהול הקשר בין מאמן למתאמנים: מחזורי הכנה, חבילה ותיעוד חיוב, העדפות ותזכורות.

עקרונות עבודה בטבלאות

1. **עמודות נגזרות על activities**. ארבע העמודות שנגזרו מהמקטעים הן מטמון של פונקציה על מידע שכבר אין לנו, ולכן לצידן מספר גרסה – שורה בגרסה ישנה מנותחת מחדש בסנכרון הבא.

2. **מי קבע את האימון**. לכל אימון יש origin – המנוע רשאי לשנות מה שהוא בנה, לא החלטה של מאמן.

3. **מחזור על שורת הקשר, ותוכנית שזוכרת את מקורה**. השיוך למחזור הוא עמודה על coach_athletes, ולתוכנית שנבנתה ממחזור יש cycle_id – כך סגירת מחזור נוגעת רק במה שנבנה ממנו.

מה עמד מאחורי כל אחת מההחלטות, ומה קרה לפני שהתקבלו – מסמך תכנון טכני, סעיף 3.

טבלה	מה יש בה
פעילות ונתונים	
profiles	שורה לכל משתמש: תפקיד, שם, גיל, מין, גובה, משקל, ספי דופק, תמונה, קוד מאמן
activities	ריצה. סיכום + ארבע עמודות נגזרות מהמקטעים: צורת קצב, שיאים, סחיפת דופק, ונקודת תחילת הסחיפה
recovery_signals	שינה, דופק מנוחה ו-HRV – יום ליום
readiness_snapshots	תמונת מצב יומית: כושר, עייפות, טריות, יחס עומס, ציון מוכנות והסבר
provider_connections	חיבור המשתמש ל-intervals.icu, כולל סטטוס ושגיאה אחרונה
תוכנית	
goal_races	מרוץ היעד – מרחק, תאריך, זמן מטרה
training_plans	תוכנית – למרוץ יעד, או בלי מרוץ כשהרץ כתב אותה בעצמו. שומרת שם, ואת המחזור שממנו נבנתה
plan_workouts	אימון בודד: יום, סוג, מרחק, קצב, סטטוס, מי קבע אותו, מה היה לפני התאמה, ולמה
plan_templates	שלד הפאזות והתמהיל השבועי – ברירת מחדל, ולכל מאמן משלו
מאמן	
coach_athletes	הקשר מאמן ↔ מתאמן, והמחזור שהרץ משובץ אליו (cycle_id). הטבלה הזו היא כל גבול האבטחה של צד המאמן
coach_cycles	מחזור הכנה – שם, מרחק, תאריך מרוץ, תבנית. "חצי מרתון תל אביב" הוא שורה כאן
subscriptions · billing_events	חבילה לכל תפקיד – שורה למשתמש ולתפקיד (scope): לרץ בסיס / פרימיום עתידי, למאמן בסיס (עד 5 מתאמנים) / פרימיום – ותיעוד כל שינוי בה. נכתבות מלשונית החיוב בהגדרות; הסליקה אינה מחוברת

טבלה	מה יש בה
coach_preferences	צבעי היומן וספי תשומת הלב של המאמן
coach_reminders	פתקים פרטיים – לא נראים למתאמן שהם עליו

3. עמודי האפליקציה

אתלט

נתיב	מה יש בו
dashboard/	מוכנות, כושר/עייפות/טריות, האימון הבא, שבוע התוכנית, לוח שנה, נפח, ספירה לאחר עם תחזית, שיאים שמקשרים לריצה
activities/	רשימת הריצות, סטטיסטיקות, נפח שבועי, מגמת קצב
activities/[id]/	ניתוח ריצה בודדת – קצב, דופק, אזורים, סחיפה, השוואה למתוכנן
plan/	התוכנית בשתי תצוגות – רשימת שבועות, או רשת חודש. ואם אין תוכנית עדיין – שלושה נתיבים להתחלה, כולם באותו עמוד: קוד מאמן, תוכנית מוכנה של Runi עם תצוגה מקדימה, או תוכנית שהרץ כותב בעצמו.
settings/	פרופיל, ומתחתיו לשוניות: חיבורים • מאמן ותוכנית (כולל עזיבת תוכנית) • חיוב (חבילת הרץ ומוקאפ אמצעי תשלום) • חשבון ואבטחה • המספרים שלך. חשבון מאמן מופנה מכאן ל- coach/settings/
numbers/	לוח "המספרים שלך" – שלושה-עשר המדדים שהמוצר מחשב, כל אחד עם הערך של הרץ, הטווח שבו הוא נמצא, הנוסחה, ומה הוא אינו אומר, וגרף היסטוריה של שבוע עד שנה. הכתובת הישנה / settings/methodology מפנה לכאן

מאמן

נתיב	מה יש בו
coach/	יומן שנה/חודש/שבוע, דגלי תשומת לב, תזכורות, קוד הצטרפות
coach/athletes/	רוסטר עם סינון לפי מרחק ואז לפי מחזור, קצב ומוכנות; כל שורה נושאת את שם המחזור
coach/athletes/[id]/	כרטיס מתאמן – מדדים, מגמה, שבוע התוכנית עם עריכה, ריצות אחרונות
coach/cycles/	מחזורי הכנה, מקובצים לפי מרחק – יצירה מתבנית, שיבוץ והסרה של רצים, עריכת שם / תאריך / תבנית, בנייה מחדש, וסגירה באישור כפול
coach/templates/	שלד הפאזות והתמהיל השבועי
coach/settings/	פרופיל, ומתחתיו לשוניות: אימון (קוד הצטרפות, תבניות, ספי תשומת לב, צבעים) • חיבורים • חיוב (שני כרטיסי החבילות ובחירה ביניהם, ומתחתם מוקאפ אמצעי תשלום) • חשבון ואבטחה • המספרים שלך. דף ההגדרות היחיד של מאמן, בכל מצב
upgrade/	הפניה בלבד ל- coach/settings#billing/ – נשמר כדי שקישורים ישנים ימשיכו לעבוד

4. נקודות הקצה

מדוע אין API נפרד – כל הפעולות בתוך הממשק המחובר עוברות כ-Server Action: קריאת פונקציה, לא קריאת HTTP, כך שאין שכבת REST/GraphQL נוספת לתחזק. Route Handlers – נקודות קצה ב-HTTP ציבורי – קיימים רק לשלושה מקרים שחייבים להתקבל מחוץ לדפדפן המחובר: קרון, webhook, וקישור ממיל.

Route Handlers – רק למה שמגיע מבחוץ

אלה נקודות הקצה הציבוריות היחידות באפליקציה – קוד שמטרתו להתקבל מחוץ לדפדפן של משתמש מחובר: קרון, webhook, וקישור מתוך מייל. כל השאר (מה שקורה בתוך הממשק) עובר כ-Server Action, לא כנתיב HTTP. פירוט ההגנה של כל נתיב מתועד במסמך אבטחת המידע; כאן רק המיפוי.

נתיב	מי קורא	הגנה
GET /api/cron/sync-intervals	Vercel Cron, 03:00	סוד בכותרת, השוואה קבועת-זמן
POST /api/webhooks/health	אפליקציית גישור לנתוני בריאות	סוד + סכימה קשיחה
auth/callback,/auth/confirm	קישור מהמייל	החלפת קוד לסשן בצד שרת

Server Actions – כל השאר

נושא	קובץ	מה בו	נקרא בעיקר מ-
אימות	auth.ts	התנתקות	מכל עמוד מחובר (תפריט עליון)
פרופיל	profile.ts	קריאה ושמירה של הפרופיל ומרוץ היעד	settings, /dashboard/
חיבורים	providers.ts	חיבור, סנכרון וניתוק intervals.icu	settings/
אימונים	activities.ts	רשימת ריצות, ניתוח ריצה, שיאים	activities, /activities/[id], /dashboard
מוכנות	readiness.ts	סדרת המוכנות וההסבר היומי	dashboard, /numbers/
תוכנית	plan.ts	מסך התוכנית, בנייתה – לרץ, או בידי המאמן עבור רץ במחזור – עזיבה, והתאמה	plan, /coach/athletes/[id]/
תוכנית	ownPlan.ts	תצוגה מקדימה ותוכנית של Runi; תוכנית שהרץ כותב בעצמו	plan/ (כשאינ תוכנית)
מאמן	coach.ts	קוד הצטרפות, הצטרפות (מחליפה מאמן קודם) ועזיבה, רוסטר, יומן, עריכת אימון, העדפות, תזכורות	coach, /coach/athletes, /coach/athletes/[id]
מאמן	cycles.ts	מחזורים: יצירה, עריכה, שיבוץ והסרה של רצים, בנייה מחדש של תוכניות, סגירה	coach/cycles/
חיוב	billing.ts	קריאת החבילה – של הרץ ושל המאמן – ומצב הרוסטר מולה; בחירת חבילה, עם רישום ל-billing_events	coach/settings, /settings/

נושא	קובץ	מה בו	נקרא בעיקר מ-
מרוץ יעד	<code>goalRace.ts</code>	יצירת מרוץ היעד – שער המנוע לתוכנית של Runi	plan/ (כשאין תוכנית)
Ask Runi	<code>insights.ts</code>	הנתונים לעשר השאלות של Ask Runi – מהערכים שכבר חושבו	כל מסך מחובר (כפתור צף)
סנכרון	<code>sync.ts</code>	"סנכרן עכשיו" – אותו מסלול של המשימה הלילית, למשתמש אחד	settings, /dashboard/

הכלל שנאכף: קובץ `"use server"` מייצא רק פונקציות אסינכרוניות, כי כל ייצוא הוא נקודת קצה ציבורית. ייצוא של קבוע – ואפילו מערך – הפיל את הבנייה פעמיים, וזו התנהגות נכונה: היא מונעת חשיפה בשוגג.

55 פעולות בשלושה-עשר קבצים. הן לא REST: קומפוננטה קוראת לפונקציה, Next.js עושה את הקריאה ברשת, והטיפוסים משותפים לשני הצדדים.

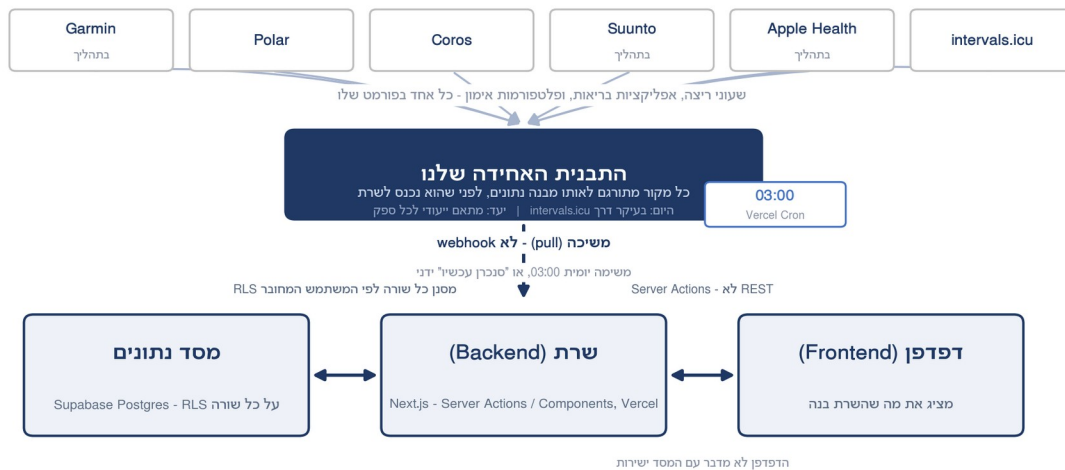
5. זרימת המידע

הדפדפן אף פעם לא מדבר עם המסד ישירות — כל קריאה עוברת קוד שרת, ו-RLS מסנן כל שורה לפי המשתמש המחובר.

נתונים נכנסים במשיכה (pull), לא בדחיפה (push) — Runi שואל את intervals.icu ביזמתו, ולא להפ

איך המידע זורם: ממקור הנתונים ועד המסך

מוצג יעד: כל מקור מתורגם לפורמט אחיד שלנו, ואז נכנס לשרת



מסך שנטען

מסך שנטען: ארבעה שלבים, ואף אחד מהם לא בדפדפן

העמוד נשלף בשרת, נבנה שם, ומגיע לדפדפן מוכן — אין שאילתה בקוד הלקוח ואין מפתח בדפדפן



הדפדפן אף פעם לא מדבר עם המסד ישירות. מה שהמשתמש לא רשאי לראות לא מגיע לשרת בכלל — הסינון קורה בשלב 3, בפוסטגריס, לא בקוד.

העמוד נשלף בשרת, נבנה שם, ומגיע לדפדפן מוכן. הדפדפן לא מדבר עם המסד ישירות — אין שאילתה בקוד הלקוח, ואין מפתח בדפדפן שמאפשר יותר ממה שהמשתמש רשאי.

נתוני אימון נכנסים

המערכת מיועדת לריבוי מקורות: Garmin (באישור מול Garmin Connect כרגע) ו-Appl Health (בפיתוח), ובעתיד גם Polar, Coros ו-Suunto — כולם יסתכנו ישירות עם האפליקציה כשהיבורים יושלמו. בשלב הנוכחי, ועד שזה קורה, intervals.icu הוא שכבת-הביניים המרכזית: הוא כבר מסונכרן מאחורי הקלעים עם רוב שיעוני הריצה בשוק, כך שהאפליקציה מדברת כיום רק עם ספק אחד. כשמקור יתחבר ישירות, ייתכנו התאמות נקודתיות בייבוא עבורו.

שלב	מה קורה
-----	---------

למה משיכה ולא webhook. התכנון המקורי היה intervals.icu — webhooks מודיע כשיש ריצה חדשה. משיכה פשוטה יותר: אין נקודת קצה ציבורית להגן עליה, אין אירועים כפולים, ואין תלות בכך שהצד השני זוכר להודיע. בפועל, רוב המשתמשים לא לוחצים כלום: המשימה הלילית רצה אוטומטית בכל לילה, והנתונים מוכנים בבוקר. המחיר הוא עד 24 שעות עיכוב באוטומטי — והמשתמש יכול ללחוץ "סנכרן עכשיו" בכל רגע לעיכוב מיידי.

איך כל פעולת-כתיבה עובדת

זה התהליך הגנרי שעומד מאחורי כל פעולה שמשנה נתונים באפליקציה — המקביל-בכתיבה ל"מסך שנטען" שמעלה.



revalidatePath הוא חובה ולא נחמדות. ב-Next.js 16, פעולת שרת שלא מבצעת אותו **לא מרנדרת את העמוד מחדש בכלל**. סנכרון שנכשל כתב "שגיאה" למסד והמסך נשאר ירוק, עם "נבדק לאחרונה" של הבוקר.

6. משתמשים והרשאות

שני סוגי הרשאה, בשתי שכבות שונות: מה מוצג בממשק, ומה מותר בפועל.

role בפרופיל — קובע מה מוצג בממשק. לא מה מותר.

RLS במסד — קובע בפועל מי יכול לקרוא ולכתוב לכל שורה, ונאכף בכל שאילתה בלי תלות בקוד.

שתי שכבות הרשאה — ורק אחת מהן באמת מגנה

role בפרופיל מחליט מה מוצג. RLS במסד מחליט מה מותר. באג בשכבה העליונה לא יכול לעקוף את התחתונה.



שני תפקידים, וחשבון אחד יכול להיות שניהם

profiles.role הוא athlete או coach. הוא מחליט מה מוצג — לא מה מותר. מאמן רואה גם את צד האתלט ("האימונים שלי"), כי מאמני ריצה רצים.

תפקיד	מה מוצג לו	הרשאות (בפועל)
athlete	רק את הנתונים והעמודים של עצמו	RLS: רואה ורושם רק שורות שבהן user_id = auth.uid().
coach	גם את עמודי המאמן, וגם את צד האתלט שלו עצמו ("האימונים שלי")	RLS: רואה ורושם שורות שבהן קיים קשר פעיל ב-coach_athletes; יכול ליצור מרוץ יעד ותוכנית לרץ שלו (מאז מיגרציה 0020); לא יכול לעדכן את שורת coach_athletes עצמה.

ההרשאה נאכפת במסד — כיסוי מלא

כל 17 הטבלאות מוגנות ב-RLS, בלי יוצא מן הכלל — כיסוי מלא. בסך הכל 44 כללים, פזורים בין הטבלאות לפי הצורך של כל אחת.

שכבה	מה היא עושה	למה לא מספיקה לבד
Middleware	מפנה מבקר לא מחובר להתחברות	רק ניתוב. לא מגן על נתונים
בדיקה בפעולה	כל פעולה בודקת (getUser())	קוד — וקוד שוכחים

שכבה	מה היא עושה	למה לא מספיקה לבד
RLS במסד	הכלל צמוד לטבלה ונאכף בכל שאלתה	—

הדפוס: אתלט רואה שורה אם `user_id = auth.uid` (). מאמן רואה אותה אם קיים קשר פעיל ב-`coach_athletes`.

וכאן הנקודה הקריטית. כל הרשאה בצד המאמן מסתכמת בשאלה "יש שורה ב-`coach_athletes`?". ולכן מדיניות ה-INSERT על הטבלה הזו היא כל גבול האבטחה. היא בדקה את העמודה הלא נכונה. ראו מסמך האבטחה.

מה מאמן רשאי לכתוב. מאז מיגרציה 0020, מאמן יכול ליצור מרוץ יעד ותוכנית עבור רץ שלו — כך נבנית תוכנית לרץ שמשובץ למחזור. את שורת הקשר עצמה (`coach_athletes`) הוא עדיין לא יכול לעדכן: היא הסכמה של הרץ בלבד.

השיבוץ למחזור עובר, לכן, דרך פונקציה צרה אחת — `set_athlete_cycle` (מיגרציה 0021):

- רצה כ-SECURITY DEFINER

- בודקת שהמאמן אכן מאמן של הרץ, ובעל המחזור

- נוגעת בעמודה אחת בלבד

- מחזירה אם שורה השתנתה בפועל, כדי ש"נוסף" לא יופיע מעל מחזור שנשאר ריק

פוסטגרס לא צועק כשהוא מסרב

מדיניות RLS שלא מתירה שורה לא מחזירה שגיאה — היא מתאימה אפס שורות והקריאה מדווחת הצלחה. לכן כל כתיבה רגישה מבקשת בחזרה את השורות שהשתנו, ואפס נרשם כסירוב ביומן השרת. מה זה עלה לנו לפני שידענו — מסמך אבטחת המידע, סעיף 2.

7. ספריות ושירותים חיצוניים

ספרייה / תשתית	תפקיד	הסבר
Next.js 16	מסגרת האפליקציה	דרישת המטלה; App Router מאפשר שרת ולקוח בפרויקט אחד
TypeScript	בטיחות טיפוסים	טיפוסי המסד נגזרים לקובץ אחד ומשמשים משני צדי השרת
Supabase	מסד, אימות ו-RLS	דרישת המטלה. נבחר גם בזכות RLS
Vercel	אחסון והרצה	דרישת המטלה. Cron בתוך אותה פריסה
supabase/ssr@	ניהול סשן	ניהול הסשן בעוגיות בין שרת ללקוח
Zod	ולידציה	ולידציה בשני הצדדים מאותה סכימה
date-fns	חשבון תאריכים	חשבון תאריכים; DST הוא מקור באגים

ספרייה / תשתית	תפקיד	הסבר
IBM Plex + Archivo Black	טיפוגרפיה	גופנים מקומיים – בלי בקשה לשרת חיצוני
Resend	שליחת מיילים	המוצר לא שולח מיילים בעצמו – Supabase Auth שולח אימות מייל ושחזור סיסמה, ו-Resend מחובר כ-SMTP מותאם. מסיר את מגבלת 2 המיילים בשעה של ברירת המחדל, בלי להקים שרת SMTP.

ומה לא נכנס, בכוונה

מה	למה לא
ספריית גרפים	כל הגרפים כתובים ידנית ב-SVG – שליטה מלאה במראה, ואפס משקל נוסף בדפדפן.
ניהול state גלובלי (Redux / Zustand)	מקור האמת הוא המסד; העמודים נבנים בשרת. useState רק למה שהוא באמת מקומי – טאב פתוח, טופס בעריכה.
ספריית UI מוכנה	העיצוב נבנה מאפס על אסימוני צבע, כי המראה הוא בידול מוצרי (ראו אפיון מוצר סעיף 6) ולא משהו שמייבאים.
מודל שפה (LLM) – בגרסה הנוכחית	ההסבר והתשובות של Ask Runi מחושבים ולא נוצרים (ראו אפיון מוצר סעיף 8). הגרסה הבאה מחברת את Ask Runi ל-Grok מעל השכבה הזו: המודל מנסח, הפונקציות מחשבות.
Supabase Branching (חיבור ל-GitHub)	מריץ מיגרציות אוטומטית בכל מיזוג – עולה כסף בלי תקרה, ונועד לזרימת-PR-ים שאין לנו (ענף יחיד). יקרה כשהאפליקציה תתפתח ונשקיע בה כסף.
Supabase CLI	מריץ מיגרציות בפקודה אחת מהמחשב, עם מעקב מסודר. נדחה לשלב זה כי טבלת המעקב שלו ריקה (24 המיגרציות רצו ידנית) ודורשת סנכרון חד-פעמי (migration repair) לפני שימוש – ייעשה כצעד תחזוקה אחרי ההגשה.
אזור זמן לכל משתמש	המוצר מניח אזור זמן ישראל אחד לכל המוצר; אם יתרחב לחו"ל, זה יהפוך לעמודה על profiles – הרחבה נדרשת רק אם וכשיהיה צורך אמיתי.

מסמך 3 – תכנון טכני מפורט

תוכן עניינים

2.....	עקרונות התכנון
2.....	1. מבנה התיקיות
3.....	2. הקומפוננטות המרכזיות
4.....	3. מבנה בסיס הנתונים
4.....	עמודות נגזרות ומספר גרסה
5.....	מי קבע את האימון
6.....	מחזור, ותוכנית בלי מרוץ
6.....	4. פעולות CRUD מרכזיות
7.....	5. תיאור ה-API
7.....	GET /api/cron/sync-intervals – המשימה היומית
7.....	POST /api/webhooks/health – נתוני התאוששות
8.....	auth/callback/-i auth/confirm/
8.....	Server Actions
8.....	6. הלוגיקה העסקית
8.....	חבילות המאמן
8.....	לוגיקת המוצר -
8.....	מודל העומס והמוכנות
9.....	יצירת תוכנית
9.....	מנוע ההתאמה
9.....	ההסבר
9.....	השאלות
10.....	היסטוריית המדדים
10.....	7. ניהול State
10.....	8. טיפול בשגיאות
10.....	הכלל: פעולה מחזירה תשובה, לא זורקת
11.....	שלוש מלכודות אמיתיות
11.....	ומה שהמשתמש רואה כשבכל זאת נשבר
11.....	וכשל ברשת אינו "אין נתונים"
11.....	9. ולידציות של קלטים
11.....	שלוש שכבות
12.....	דוגמאות
12.....	10. חוויית המשתמש המרכזית
12.....	הכלל שנאכף בכל מסך
12.....	מה שנגזר מזה

עקרונות התכנון

- **פונקציות טהורות ב-lib/**, **לא בקומפוננטות** – כל חישוב עסקי (עומס, מוכנות, התאמת תוכנית, ניתוח ריצה) יושב בקוד שלא יודע ש-React או Supabase קיימים; מקבל מספרים ומחזיר מספרים, ונבדק בלי מסד וכלי דפדפן.
- **Server Components כברירת מחדל** – עמודי /app שולפים בשרת ומעבירים למסך מוכן; "use client" מסומן רק על מה שבאמת מגיב לקליק.
- **פעולה מחזירה תשובה, לא זורקת** – כל פעולת כתיבה מחזירה תוצאה מפורשת של הצלחה או כישלון – ולא נשענת על שכבת try/catch גלובלית שתתפוס את מה שנשכח.
- **ולידציה בשלוש שכבות** – קלט נבדק בדפדפן לנוחות, ב-Server Action בוודאות, ובמדיניות המסד כרשת ביטחון אחרונה שלא ניתן לעקוף מהלקוח.
- **בלי נתון אמיתי – לא מציגים מספר** – קו מקף, מצב ריק, או הסתרה. הכלל נאכף בכל מסך, אחרי שסריקה שיטתית מצאה 47 הפרות ממנו. ראו סעיף 10.

1. מבנה התיקיות

מה יש שם	נתיב
/app – עמודים ונקודות קצה (App Router)	
	login, signup/(auth)
אישור מייל ושחזור סיסמה	auth/callback, confirm, reset
	dashboard, plan, activities, activities/[id], numbers, settings, onboarding
בית, מתאמנים, מתאמן, מחזורים, תבניות, הגדרות	/coach
שער התפקיד – נבדק פעם אחת לכל הענף	coach/layout.tsx
cron/sync-intervals, webhooks/health	/api
שגיאה ו-404 בעיצוב המוצר	error.tsx, not-found.tsx
/components – רכיבי תצוגה בלבד	
מסך שלם: מקבל נתונים, לא שולף	/screens
	/dashboard/, coach/, plan/, settings/, ui
"use server" – /actions/, הגבול בין לקוח לשרת	
/lib – לוגיקה טהורה, בלי React, בלי מסד, נבדקת	
עומס, ACWR, יצירת תוכנית, תוכנית של הרץ עצמו, התאמה, ספים	/planning
הצינור שמחשב תמונת מצב יומית	/readiness

מה יש שם	נתיב
אזורים, חישובים וגיאומטריית גרפים	/activity
דיבור עם intervals.icu וייבוא	/wellness/, providers
רוסטר, יומן, מחזורים, תבניות	/coach
חבילות מאמן ומושבים — plans.ts	/billing
הרכבת נתונים לתצוגה + הטקסטים	/dashboard/, screens
בניית ההסבר	/narrative
עשר השאלות ש-Ask Runi עונה עליהן	/insights
הכללים מאחורי ערכת הרכיבים — עמודות, תא יום, רשת חודש	/ui
קריאת התפקיד, במקום אחד לכל מי שצריך אותו	/auth
שבוע, אזור זמן — מקור אמת אחד	/time
סכימות Zod	/validation
middleware-i /types	
טיפוסי המסד — מקור אחד לשני הצדדים	types/database.types.ts
הפניה של מבקר לא מחובר	middleware.ts

הכלל שמארגן הכל: /lib לא יודע ש-React קיים ולא יודע ש-Supabase קיים. הוא מקבל מספרים ומחזיר מספרים. לכן 835 הבדיקות רצות בלי מסד ובלי דפדפן, בפחות מחצי דקה.

/actions מדבר עם המסד וקורא ל-components- /lib/. מציג מסך לא שולף בעצמו — הוא מקבל נתונים כ-props.

2. הקומפוננטות המרכזיות

עמודי /app הם Server Components — שולפים בשרת ומעבירים למסך. רק מה שמגיב לקליק מסומן "use client".

קומפוננטה	תפקיד
DashboardView	הדשבורד המלא: מוכנות, כושר/עייפות/טריות, אימון הבא, שבוע, לוח שנה, שיאים
ReasoningPanel	פירוק ציון המוכנות — מה נמדד, כמה שקל, מה זה אומר
ActivitiesView	רשימת ריצות, נפח שבועי, מגמת קצב
ActivityDetailView	ניתוח ריצה — קצב, דופק, אזורים, סחיפה, השוואה למתוכנן
PlanView	לוח חודשי של התוכנית עם ציר שלבים (base/build/peak/taper); יום נפתח בלחיצה

קומפוננטה	תפקיד
PlanStart	המצב הריק של plan/ – שלושה נתיבים באותו מסך: קוד מאמן, תוכנית של Runi עם תצוגה מקדימה (פאזות, שבוע לדוגמה, שיאים), ותוכנית משלו עם עורך שבוע
SettingsView	פרופיל, ומתחתיו לשוניות: חיבורים • מאמן ותוכנית (כולל עזיבה) • חיוב • חשבון ואבטחה • המספרים שלך. חשבון מאמן מופנה ממנו לדף ההגדרות של המאמן, שבנוי מאותן לשוניות
AvatarEditor	בחירת תמונה, הזזה וזום – ושמירת חיתוך ריבועי
CoachHomeView	יומן, דגלי תשומת לב, תזכורות, קוד
CoachCalendar	שנה / חודש / שבוע
CoachAthleteView	כרטיס מתאמן – שבוע של שבעה ריבועים, אחד נפתח בלחיצה
Avatar	תמונה או ראשי תיבות, בכל מקום באתר
/components/ui	ערכת הרכיבים המשותפת – אריח מדד, גרף עמודות, תא יום, צ'יפ, כותרת מקטע, ספירה לאחור, מצב ריק, ונוסחה
ComparePanel	השוואת עד שלוש ריצות – קצב ודופק על אותו ציר
NumbersView	לוח המספרים – שלושה-עשר אריחים, ופאנל שנפתח לכל מדד: הערך, הטווח, הנוסחה בסדר-דפוס, וגרף היסטוריה עם טווחי שבוע / חודש / שלושה חודשים / שנה
CoachModeBar	מתג מאמן ↔ מתאמן, ומשפט שאומר של מי המספרים על המסך
CoachTemplatesView	תבנית כצורה: שבוע 1 עד N, עם ספירת המתאמנים בכל שבוע
CoachCyclesView	מחזורים מקובצים לפי מרחק. מחזור מנוהל: חברים עם "שבוע N מתוך M", שיבוץ והסרה, עריכה, בנייה מחדש, וסגירה באישור כפול
CoachPlanCards • PaymentMethodMock	בסיס או פרימיום – שני כרטיסים באותה צורה של נתיבי התוכנית, בתוך לשונית החיוב בהגדרות; החבילה הנוכחית מסומנת, בחירה נשמרת בלי לעזוב את הדף, ומתחת לפרימיום כתוב שאין חיוב בגרסה זו. מתחתם טופס אמצעי תשלום – מוקאפ מסומן ככזה, שאינו שומר ואינו מחייב
InsightsPanel	פאנל Ask Runi – רשימת שאלות שניתן לסנן, ותשובה עם טבלה וגרף לכל אחת
AskRuniLauncher	הכפתור הצף שמלווה את הרץ בכל המסכים – מתקפל בגלילה, ופותח את InsightsPanel

דוגמה לתיקון שנבע מהמבנה הזה: היו חמישה מימושים שונים של תמונת פרופיל, כל אחד עם נפילה-לאחור אחרת – קובץ שלא קיים, סמילי, המילה "Photo", וראשי תיבות בלי עיגול. אותו משתמש נראה כארבעה אנשים שונים. Avatar אחד פתר את זה, וזו בדיוק הסיבה שקומפוננטות משותפות שוות את המחיר.

3. מבנה בסיס הנתונים

24 מיגרציות. הרשימה המלאה במסמך הארכיטקטורה, כאן שלוש ההחלטות מרכזיות-

עמודות נגזרות ומספר גרסה

activities מחזיקה, לצד הסיכום, ארבע עמודות שנגזרו מהמקטעים הגולמיים:

עמודה	טיפוס	מה זה
pace_shape	jsonb	צורת הקצב, לגרף הזעיר
best_efforts	jsonb	הזמן הטוב ביותר בכל מרחק
cardiac_drift_pct	numeric	כמה הדופק נסחף בעוד הקצב נשמר
drift_onset_m	integer	באיזה מטר זה התחיל
streams_derived_version	smallint	גרסת הפונקציה שחישבה את השורה – מזהה שורות מיושנות
source_updated_at	timestampz	מתי הריצה עצמה עודכנה אצל הספק

המקטעים עצמם נזרקים – עשרות מגה-בייט לשנה. אז ארבע העמודות האלה הן **מטמון של פונקציה על מידע שכבר אין לנו**, וזה בדיוק מה שהופך אותן למסוכנות.

הבעיה שקרתה בפועל: חישוב השיאים מדד זמן שעון ולא זמן תנועה, אז רמזור אדום בתוך 10 ק"מ מהיר נספר לרעת הריצה. ריצה שהייתה **43 שניות מהירה יותר** נרשמה כשיא האיטי יותר. תיקון הפונקציה לא תיקן שורה אחת שכבר נשמרה.

הפתרון – מספר גרסה. המייבא יודע איזו גרסה הוא מיישם. השאילתה מחפשת `streams_derived_version < DERIVATION_VERSION`. משנים נוסחה, מעלים מספר, והנתונים מתקנים את עצמם בסנכרון הבא.

ו-source_updated_at עונה על החצי השני: מה אם הריצה השתנתה? אתלט חותך ריצה ב-intervals.icu – המרחק מתעדכן, והניתוח נשאר של הקובץ הישן לנצח. שמירת חותמת השינוי של הספק הופכת את "השתנה?" לשאלה שאפשר לענות עליה.

מי קבע את האימון

עמודה	מה זה
origin	מי קבע את האימון – generated / coach / athlete
planned_distance_original	המרחק המקורי לפני התאמה – כדי שאפשר יהיה להחזיר
adjusted_reason	הסיבה להתאמה, במילות המתאמן
adjusted_at	מתי בוצעה ההתאמה

שני באגים עם שורש אחד: status עשה עבודה שהוא לא מתאים לה.

מנוע ההתאמה מקטין ל-80% וכותב `status = 'adjusted'`, ומדלג על כל מה שאינו `'planned'` – נכון, כי זה מונע הצטברות. אבל זה גם אמר **שאין דרך חזרה**. העומס חוזר לנורמה ואימוני השבוע נשארים מקוצרים לנצח.

ובמקביל: עריכת מאמן השאירה `status = 'planned'` – בדיוק המצב שהמנוע מחפש. מאמן שקבע 18 ק"מ בערב מצא 14.4 ק"מ בבוקר, בלי שום סימן.

origin אומר של מי ההחלטה. planned_distance_original שומר מה היה, כדי שאפשר יהיה להחזיר. adjusted_reason הוא הסיבה, במילים של המתאמן.

מחזור, ותוכנית בלי מרוץ

מה זה	עמודה
הפך לאופציונלי – תוכנית של הרץ עצמו אין לה מרוץ	training_plans.goal_race_id
איך קוראים לתוכנית, ומאיזה מחזור נבנתה	training_plans.name, cycle_id
הרץ משובץ למחזור על שורת הקשר עצמה	coach_athletes.cycle_id
שם, מרחק, תאריך מרוץ, תבנית	coach_cycles

מיגרציה 0020 פתחה את שלושת נתיבי התוכנית ואת המחזורים, עם המחזור כעמודה על שורת הקשר (לא טבלת קישור נפרדת) – כדי שרץ יהיה במחזור אחד בלבד אצל המאמן שלו. מלכודת אחת בדרך: מדיניות RLS שנתנה לרץ בלבד לעדכן את שורתו חסמה בשקט גם את המאמן כשניסה לשבץ למחזור – **מיגרציה 0021** פתרה עם פונקציית SECURITY DEFINER אחת שממוקדת רק בעמודת cycle_id.

4. פעולות CRUD מרכזיות

ישות	יצירה	קריאה	עדכון	מחיקה
profiles	נוצר אוטומטית בהרשמה	הגדרות, כרטיס מאמן	שמירת פרופיל	—
provider_connections	חיבור intervals.icu	פאנל ההגדרות	סנכרון, סטטוס, שגיאה	ניתוק
activities	ייבוא (upsert לפי מזהה חיצוני)	רשימה, דשבורד	ניתוח מקטעים	—
goal_races	עמוד התוכנית (הרץ), או שיבוץ למחזור (המאמן)	תוכנית, דשבורד, מחזורים	עדכון בטבלה – חוץ משינוי סוג המרוץ (למשל שיבוץ למחזור אחר), שסוגר את הישן ופותח חדש	סטטוס
training_plans + plan_workouts	אחד משלושת הנתיבים, או שיבוץ למחזור	תוכנית, דשבורד, יומן	התאמה אוטומטית או עריכת מאמן	עזיבה – סטטוס abandoned, לא מחיקה
readiness_snapshots	חישוב מחדש	דשבורד, רוסטר	upsert לפי (משתמש, תאריך)	—

ישות	יצירה	קריאה	עדכון	מחיקה
coach_athletes	המתאמן מקליד קוד	רוסטר	סטטוס (הרץ); שיבוץ למחזור דרך set_athlete_cycle (המתאמן)	שני הצדדים רשאים
coach_cycles	המתאמן, מתבנית	מסך המחזורים, רוסטר	שם, תאריך, תבנית	אחרי סגירת תוכניות החברים וריקון המחזור
subscriptions	בחירת חבילה בלשונית החיוב	הגדרות המתאמן	החלפת חבילה) upsert לפי (משתמש)	—

שתי הערות:

מרוץ יעד מתעדכן במקום כששינוי הוא בתאריך או בזמן היעד — אחרת שינוי תאריך היה מייצג תוכנית שרצים לפיה כבר חודש. שינוי סוג המרוץ עצמו (בעיקר בשיבוץ מחדש למחזור) סוגר את הישן ופותח חדש.

בניית תוכנית מסרבת לדרוס. יש כבר תוכנית? הפעולה מחזירה סירוב מנומק. מי שרוצה אחרת עוזב קודם — והעזיבה סוגרת את התוכנית, לא מוחקת אותה, כי השבועות שרצו לפיה הם היסטוריה.

5. תיאור ה-API

נקודת קצה	מי קורא	הגנה	מה עושה
GET /api/cron/sync-intervals	Vercel Cron, 03:00	סוד קבוע-זמן) timingSafeEqual	לכל חיבור פעיל: ייבוא, ניתוח מקטעים, מוכנות, התאמת תוכנית
POST /api/webhooks/health	ספק חיצוני	סוד + סכימת Zod קשיחה	כותב נתוני התאוששות; ערך לא סביר נדחה
auth/callback, /auth/confirm	קישור מהמייל	PKCE — קוד חד-פעמי מוחלף לסשן בשרת	השלמת הרשמה/שחזור סיסמה; next נבדק ולא נסמך עליו
Server Actions	רכיבי לקוח	getUser() בכל פעולה	55 פעולות — כל ייצוא הוא נקודת קצה ציבורית

המשימה היומית — GET /api/cron/sync-intervals

מי קורא: Vercel Cron, 03:00. **הגנה:** סוד ב-Authorization, בהשוואה קבועת-זמן (timingSafeEqual) — השוואת מחרוזות רגילה נשברת מוקדם ומדליפה את אורך ההתאמה. **מה עושה:** לכל חיבור פעיל — ייבוא, ניתוח מקטעים, חישוב מוכנות, התאמת תוכנית. כישלון אצל משתמש אחד לא עוצר את השאר.

נתוני התאוששות — POST /api/webhooks/health

הגנה: סוד + סכימת Zod קשיחה. ערך לא סביר נדחה ולא נכתב.

`auth/callback/-! auth/confirm/`

למה הם קיימים: `supabase/ssr@` משתמש ב-PKCE. הקישור במייל מגיע עם קוד חד-פעמי שצריך להחליף **לשון בצד השרת**, כדי שהעוגייה תיכתב בתשובה שהדפדפן ישמור.

מה קרה בלי זה: חשבון קיים ובלתי שמיש — ראו מסמך אבטחת המידע, סעיף 1.

next נבדק ולא נסמך עליו — כתובת מלאה שם הופכת את זה להפניה פתוחה.

Server Actions

55 פעולות. **כל ייצוא מקובץ "use server" הוא נקודת קצה ציבורית**, ולכן: רק פונקציות אסינכרוניות מיוצאות, וכל אחת בודקת `getUser()` לפני כל גישה למסד. כישלון לא צפוי נרשם ב-`console.error` בשרת — וביומן של Vercel — לפני שהוא הופך להודעה למשתמש.

6. הלוגיקה העסקית

המודל העסקי עצמו פשוט: שתי חבילות לרץ ושתיים למאמן, מושבים מוגבלים בבסיס של המאמן, וטופס תשלום שהוא עדיין מוקאפ. שאר הסעיף הזה הוא לוגיקת המוצר שתומכת בזה — איך מחושבות התוכנית והמוכנות שהחבילה בעצם מוכרת.

חבילות המאמן

`lib/billing/plans.ts` מחזיק את שתי חבילות המאמן — בסיס (חינם לחצי שנה ואז \$5 לחודש, 5 מושבים) ופרימיום (\$10 לחודש, ללא הגבלה) — ואת שתי חבילות הרץ (בסיס חינם עם כל היכולות; פרימיום עם \$10 RunAI, 10\$ לחודש, מוצגת כ"בקרוב" בלי כפתור רכישה) — ואת המיפוי לערכים שטבלת `subscriptions` מכירה מאז 0001 (free / pro). מושב הוא מתאמן פעיל ברוסטר. הכלל "יש עוד מושב?" הוא פונקציה טהורה עם בדיקות, אבל האכיפה עצמה יושבת ב-`join_coach` במסד (מיגרציה 0022): המתאמן שמקליד את הקוד לא רשאי לקרוא את המנוי של המאמן, ולכן רק פונקציה שרצה כ-`definer` יכולה לספור מולו. חשבון מאמן בלי שורת מנוי — מלפני החבילות — אינו נחסם לעולם; לשונית החיוב מציגה לו "בחר חבילה" עד שיבחר.. מיגרציה 0023 מוסיפה `scope` לשורת המנוי, כך שמאמן שגם רץ מחזיק שורה לכל תפקיד — ובחירת חבילה בלשונית החיוב כותבת רק את השורה של התפקיד שבו היא נעשתה.

לוגיקת המוצר -

מודל העומס והמוכנות

· ריצה → עומס יומי → כושר / עייפות / טריות → יחס עומס → ציון מוכנות

עומס לאימון בודד. כשיש דופק — `Banister TRIMP`, שמשקלל משך בעצימות עם עקומה מעריכית, כי עשרים דקות בדופק גבוה אינן פי שניים מעשר. כשאין דופק — נופלים לקצב מותאם-שיפוע (`Minetti GAP`), כי קילומטר בעלייה אינו קילומטר במישור.

כושר, עייפות וטריות (CTL / ATL / TSB). שני ממוצעים נעים מעריכיים — 42 יום לכושר, 7 לעייפות — וההפרש ביניהם הוא הטריות.

יחס עומס (ACWR). שבוע אחרון חלקי הממוצע של ארבעה. מעל 1.5 = סיכון. מיושם בגרסה עם **תיקון הטיה** – ממוצע מעריכי בתחילת סדרה מוטה כלפי מטה, ובלי תיקון האתלט נראה בטוח יותר משהוא בשבועות הראשונים.

ציון מוכנות 0-100 – שקלול של טריות, יחס עומס, שינה, HRV וסחיפת דופק. **לכל רכיב יש ציון משנה, משקל, ומשפט הסבר** – וזה מה שפאנל ההסבר מציג.

יצירת תוכנית

מחלק את השבועות עד המרוץ לארבע פאזות. **הנפח נגזר מהיכולת הנוכחית של הרץ** ולא מטבלה: הריצה הארוכה גדלה מהריצה הארוכה שלו היום, עם שבוע הקלה כל רביעי.

התבנית של המאמן נכנסת כפרופורציות ולא כלוח זמנים. תבנית של 14 שבועות שניתנת למי שרץ בעוד תשעה הופכת לתשעה שבועות ששומרים על הצורה – ולא לתוכנית שנגמרת אחרי המרוץ.

סירוב אחד מפורש, והשלמה אחת. מרוץ קרוב מדי – הפעולה מסרבת ומסבירה. רץ בלי היסטוריית ריצות כבר לא נדחה: הוא מתבקש לשני מספרים – נפח שבועי וריצה ארוכה אחרונה – והם תופסים את מקום היכולת שנקראת מהריצות; הקצבים נגזרים מזמן היעד דרך Riegel. רץ שמאמן משבץ למחזור בלי ריצות מקבל ברירת מחדל זהירה (20 ק"מ בשבוע, ארוכה של 8).

תצוגה מקדימה לפני יישום. previewRuniPlan מריץ את אותו מחולל בלי לכתוב, ומחזיר את האורך, הפאזות, שבוע אחד מאמצע שלב הבנייה – שם התוכנית הכי דומה לעצמה – ואת השיא שאליו היא מטפסת. תוכנית שמאשרים בעיוורון היא תוכנית שעוזבים אחרי שבוע.

תוכנית של הרץ עצמו. buildOwnPlan פורש דפוס שבועי על N שבועות (1-24) משבוע שמתחיל ביום ראשון. עלייה הדרגתית, אם ביקשו: $1.07 \times$ לשבוע, ושבוע הקלה של $0.75 \times$ כל רביעי. אין מרוץ – ולכן הספירה לאחר בבית היא לסוף התוכנית, לא ליום מרוץ.

תחזית Riegel. מהשיא הארוך ביותר של הרץ: $T_2 = T_1 \cdot (D_2/D_1)^{1.06}$. מוצגת לצד הספירה לאחר, עם סימון האם זמן היעד בהישג יד. תחזית, לא הבטחה – והמסך אומר את זה.

מנוע ההתאמה

מקטין כשיחס העומס חוצה סף, או כששיעור גבוה של סחיפת דופק. **מדלג** על מה שאדם קבע (origin). **מחזיר** כשהסיבה חלפה – הפעולה היחידה שיכולה רק להוסיף מרחק בחזרה.

ההסבר

buildNarrative **מרכיב** משפט מהערכים שכבר חושבו. בגרסה הנוכחית אין מודל שפה, ולכן אין מספר שאפשר להמציא, וכל טענה במשפט ניתנת למעקב עד לחישוב. הגרסה הבאה מחברת את Ask Runi ל-Grok – מעל השכבה הזו: המודל מזהה את השאלה ומנסח, הפונקציות ממשיכות לחשב.

השאלות

lib/insights/questions.ts מחזיק עשר שאלות, כל אחת פונקציה טהורה שמקבלת את נתוני הרץ ומחזירה תשובה. getInsightData שולף את החבילה פעם אחת בפתיחת הפאנל – מי שלא פותח אותו לא טוען דבר, ומעבר בין השאלות לא עולה קריאה נוספת. שאלה שאין לה מספיק נתונים מחזירה insufficient ומשפט שמסביר מה חסר, ויש בדיקה למקרה הזה בכל אחת מעשר. הפאנל מסנן את רשימת השאלות ואינו מקבל טקסט חופשי, כדי שכל מה שמוצג אכן נענה.

היסטוריית המדדים

buildHistory ב-lib/screens/numbers.ts בונה לכל מדד סדרה לפי המקור שלו:

- **מדדי תמונת המצב** – כושר, עייפות, טריות, יחס עומס, מוכנות: מתוך readiness_snapshots, בטווחי שבוע / חודש / שלושה חודשים.
- **מדדי ריצה ולילה** – דופק, קצב, סחיפה, TRIMP, התאוששות: מהריצות והלילות עצמם, עד שנה.
- **נפח** – כעמודות: יומיות בשבוע, שבועיות מעבר לו; העמודה האחרונה היא השבוע הרץ, ומסומנת ככזו. קצב מותאם-שיפוע ותחזית אין להם היסטוריה, והפאנל אומר זאת במקום לצייר קו ריק.

7. ניהול State

שכבה	איפה	דוגמה
מקור האמת	Supabase	ריצות, תוכנית, מוכנות
מצב שרת	Server Components	העמוד נבנה בשרת עם הנתונים
מצב מקומי	useState	טאב פתוח, יום נבחר, טופס בעריכה
רענון	revalidatePath	אחרי כל כתיבה

אין state גלובלי בלקוח. אין Redux ואין Zustand – אין מה לסנכרן, כי הנתונים לא חיים בלקוח מלכתחילה.

מלכודת שנתקלנו בה: `useState(props.x)` נותן העתק שנקבע פעם אחת ולא מתעדכן. מסנן הקצב ברוסטר עשה בדיוק את זה – "נקה" איפס את המסנן והשאיר את הערכים בתיבות, ומסך הצהיר שהוא מסונן כשלא היה.

8. טיפול בשגיאות

סוג שגיאה	דוגמה	איך מטופל
ולידציית קלט	מרוץ יעד בלי תאריך	הודעה קריאה למשתמש; לא נשמר
סירוב הרשאה שקט (RLS)	עריכת מאמן שלא עברה	נבדק ע"י קריאת השורות שהשתנו – אפס = סירוב
"הצלחה" שהיא כישלון	סנכרון נכשל בלי revalidate	revalidatePath מחויב אחרי כל כתיבה
שגיאה לא צפויה	קריסה ברינדור	error.tsx / not-found.tsx בעיצוב המוצר
כשל רשת זמני	תקלת שרת בהורדת מקטע	השורה נשארת בתור, לא מסומנת כמנותחת

הכלל: פעולה מחזירה תשובה, לא זורקת

```
• type Result<T> = { ok: true; data: T } | { ok: false; error: string }
```


הודעת שגיאה נכתבת למשתמש, לא למפתח: "A goal race needs a date as well as a distance" ולא "null constraint violation".

שלוש מלכודות אמיתיות

סירוב שקט של הרשאה. RLS לא זורק — הוא מתאים אפס שורות. הפתרון: מבקשים בחזרה את השורות שהשתנו; אפס = סירוב (ראו מסמך אבטחת המידע, סעיף 2).

כישלון שנראה כהצלחה. סנכרון שנכשל כתב "שגיאה" למסד — ולא ביקש רינדור מחדש. ב-Next.js 16 פעולה שלא מבצעת `revalidatePath` לא מרנדרת בכלל, אז הפאנל נשאר ירוק וכתוב "מחובר".

מטמון שהופך כפתור לשקר. בקשת נתוני ההתאוששות נשמרה במטמון לשעה, ומפתח המטמון היה יציב ליום שלם. "סנכרן עכשיו" החזיר תשובה מלפני שעה — והודיע שהסתנכרן.

ומה שהמשתמש רואה כשבכל זאת נשבר

not-found.tsx-i error.tsx נותנים לשגיאה לא צפויה ול-404 עמוד בעיצוב המוצר עם דרך חזרה, במקום המסך הלבן של Next.js. כל הודעות הפעולות באנגלית, בשפת המוצר, אחרי שנמצאו כמה בעברית שנשארו מהפיתוח.

וכשל ברשת אינו "אין נתונים"

חריגת קצב (429) או תקלת שרת בהורדת מקטע **אינה** "לריצה הזו אין מקטעים". הקוד סימן אותה כמנותחת, והשאילתה שמחפשת ממתינים כבר לא מסתכלת עליה — ריצה איבדה את הגרף ואת השיא שלה לנצח. עכשיו: תקלה זמנית זורקת והשורה נשארת בתור.

9. ולידציות של קלטים

סוג קלט	דוגמאות
מספר בטווח	גיל, גובה, משקל, מרחק אימון, מרחק מרוץ
תאריך	תאריך מרוץ, תאריך התחלת תוכנית עצמאית
ערך מרשימה סגורה (enum)	מרחק מרוץ, סוג מרוץ
קובץ	תמונת פרופיל — סוג ותקרת גודל
מבנה מורכב	תבנית מאמן, תוכנית עצמאית — פאזות/שבועות שחייבים להסתכם נכון
הפניה (redirect)	כתובת next אחרי אימות — רק נתיב יחסי

שלוש שכבות

איפה	מה
דפדפן	משוב מיידי — <code>type="email", minLength, required</code>
Server Action	השכבה שמגנה. Zod, טווחים, וזהות הבעלים
מסד	<code>check, unique</code> , מפתחות זרים, RLS-i

הכלל: בדיקה בדפדפן היא נוחות, לא הגנה – היא נמצאת אצל התוקף. כל קלט נבדק **שוב** בשרת.

דוגמאות

- גיל 10-100, גובה 100-250, משקל 30-250, מרחק מרוץ מרשימה סגורה; תאריך בפורמט מלא
- מרוץ יעד דורש **גם** מרחק **וגם** תאריך, מרחק אימון 0-100 ק"מ, תמונה – סוג ותקרת גודל, אחרי הקטנה בדפדפן
- תבנית מאמן – הפאזות חייבות להסתכם באורך התוכנית, והשבוע ב-7 ימים עם יום מנוחה וריצה ארוכה
- תוכנית של הרץ עצמו – שם, תאריך התחלה, 1-24 שבועות, שבוע בן 7 ימים עם לפחות ריצה אחת, כל ריצה 1-60 ק"מ, קצב בתבנית 5:30

10. חוויית המשתמש המרכזית

אתלט שרואה מספר שלא קרה מפסיק לסמוך על שאר המסך – ולא חוזר. חוויית המשתמש היא לא שכבת עיצוב מעל המוצר, היא כלי אמון קריטי: ברגע שמספר אחד לא אמיתי מוצג כאילו הוא אמיתי, כל שאר הנתונים במסך מוטלים בספק. מכאן הכלל שנאכף בכל מסך:

הכלל שנאכף בכל מסך

בלי נתון אמיתי – לא מציגים מספר. קו מקף, מצב ריק, או הסתרה.

נאכף אחרי שסריקה שיטתית מצאה 47 הפרות שכולן משורש אחד – נתוני-דוגמה שנשארו כברירות מחדל (מסמך אפיון בדיקות, סעיף 8).

מה שנגזר מזה

מצב ריק שאומר מה לעשות – לא קופסה ריקה. "אין תוכנית" מגיע עם שלוש דרכים להתחיל; בית בלי ריצות אבל עם תוכנית אומר "התוכנית מוכנה, חבר את השעון" ולא מציע לבנות אחת.

כפתור שלא עושה כלום – לא קיים. נמצאו ארבעה.

כישלון נראה. אם המסד אומר "שגיאה", המסך אומר "שגיאה".

וכל גרף מטופל במקרי הקצה: נקודה אחת, אפס נקודות, כל הערכים זהים, וטווח מנוון – כולם מצוירים או מוסתרים, ולא הופכים ל-NaN בתוך path.

מסמך 4 – אפיון בדיקות

תוכן עניינים

2.....	מה נבדק, ולמה דווקא כך
2.....	מה בכוונה לא נבדק אוטומטית.....
3.....	1. בדיקות לפיצ'רים המרכזיים.....
3.....	המודל הפיזיולוגי – הלב של המוצר.....
3.....	התוכנית.....
4.....	הדשבורד והמסכים.....
5.....	צד המאמן.....
5.....	2. בדיקות לקלטים לא תקינים.....
6.....	3. בדיקות לתהליכים עסקיים.....
6.....	4. בדיקות הרשאות.....
7.....	5. בדיקות למסד הנתונים.....
7.....	6. בדיקות למקרי קצה.....
7.....	זמן ואזור זמן – 24 בדיקות.....
8.....	גרפים – 63 בדיקות.....
8.....	נתונים חסרים.....
8.....	7. בדיקות UI.....
8.....	8. בדיקות ידניות מתועדות.....
8.....	שלושה סבבים.....
9.....	סבב ידני לפני הגשה.....
9.....	9. מגבלות ומה היה נבדק בהמשך.....

מה נבדק, ולמה דווקא כך

תשעה סוגי בדיקות מכסים את המוצר. הטבלה ממפה כל סוג לסעיף שמפרט אותו – ולוגיקת הבחירה בגישה הזו מוסברת בהמשך.

סוג בדיקה	סעיף
בדיקות לפיצ'רים המרכזיים	1
בדיקות לקלטים לא תקינים	2
בדיקות לתהליכים עסקיים	3
בדיקות הרשאות	4
בדיקות למסד הנתונים	5
בדיקות למקרי קצה	6
בדיקות UI	7
בדיקות ידניות מתועדות	8
מגבלות ומה היה נבדק בהמשך	9

835 בדיקות אוטומטיות ב-57 קבצים, שרצות בפחות מחצי דקה – בלי מסד נתונים ובלי דפדפן.

זה אפשרי בגלל החלטה ארכיטקטונית: כל הלוגיקה יושבת ב-[lib/](#), שלא יודע ש-React קיים ולא יודע ש-Supabase קיים. הוא מקבל מספרים ומחזיר מספרים.

ולמה זה חשוב: בדיקה שרצה בחצי דקה רצה לפני כל שינוי. בדיקה שדורשת מסד ודפדפן רצה פעם ביום, ואז מפסיקים להריץ אותה.

מה בכוונה לא נבדק אוטומטית

רינדור קומפוננטות ו-E2E בדפדפן. תוכננו React Testing Library ו-Playwright ולא מומשו. **הסיבה כנה:** התקציב הוגבל, והיה צריך לבחור בין לבדוק שכפתור מרונדר לבין לבדוק שהחישוב מאחוריו נכון.

בחרנו בחישוב – כי באג בכפתור נראה בעין, ובאג בנוסחת העומס לא. ריצה שנרשמה כשיא איטי יותר משהייתה היא באג שאף אחד לא רואה, וזה בדיוק סוג הבאג שבדיקה תופסת.

במקום זאת בוצעה סריקה ידנית מתועדת מקצה לקצה – ראו סעיף 8. המטלה מאשרת בדיקות ידניות מתועדות במפורש.

1. בדיקות לפיצ'רים המרכזיים

המודל הפיזיולוגי – הלב של המוצר

מה יכול להשתבש:

- חישוב עומס שגוי כשדופק חסר, ואין נפילה נקייה לקצב מותאם-שיפוע
- תיקון ההטיה בתחילת סדרה לא מיושם – ACWR מטעה בשבועות הראשונים
- ציון מוכנות ששוקלל רכיב חסר כאילו הוא אפס במקום להסביר שהוא חסר

מה נבדק	כמה	דוגמאות
חישובי אימון בודד	46	קצב מותאם-שיפוע, מדדים נגזרים
כושר / עייפות / טריות	11	ממוצעים מעריכיים, התכנסות, סדרה ריקה
יחס עומס (ACWR)	17	חישוב נכון, תיקון הטיה בתחילת סדרה, סף סיכון
עומס לאימון	14	TRIMP עם דופק; נפילה לקצב בלי דופק
ציון מוכנות	11	שקלול, טווח 0-100, רכיבים חסרים
סחיפת דופק	4	זיהוי עלייה בקצב קבוע, ונקודת ההתחלה
ספי דופק	—	מדוד מול משוער, וריכוך שלא קופץ משבוע קשה

התוכנית

מה יכול להשתבש:

- מנוע ההתאמה נוגע במה שהמאמן קבע ידנית במקום לדלג
- תוכנית לא מסתיימת ביום המרוץ בפועל
- תבנית מאמן מיושמת כלוח זמנים קשיח במקום כפרופורציות
- קצבים מחושבים מטבלה סטטית במקום מזמן היעד (Riegel)

מה נבדק	כמה	דוגמאות
יכולת הרץ	23	ריצה ארוכה שגדלה מהקיים, שבוע הקלה כל רביעי

מה נבדק	כמה	דוגמאות
יצירת תוכנית	4	חלוקה לפאזות, אורך נכון עד המרוץ
תבנית מאמן	10	פרופורציות ולא לוח זמנים; ריצה ארוכה בשישי; מנוחה בשבת
מנוע ההתאמה	4	הקטנה כשהעומס חוצה סף
מי קבע את האימון	8	המנוע לא נוגע במה שמאמן קבע; החזרה כשהסיבה חלפה; לא מעמיק הקטנה קיימת
מבנה האימון	8	נבנה מהמרחק והקצב של האימון עצמו
תוכנית של הרץ עצמו	6	ולידציה; פריסה משבוע שמתחיל ביום ראשון; עלייה של 7% והקלה כל רביעי; קצבים מזמן היעד (Riegel)
חבילות המאמן	4	מיפוי בסיס/פרימיום לערכי המסד; מושב חמישי נחסם ושישי לא; חשבון בלי חבילה לא נחסם; תווית הרוסטר מול המכסה

הדשבורד והמסכים

מה יכול להשתבש:

- גרף שמצייר NaN בתוך מסלול SVG במקום להסתיר נקודה חסרה
- דגימה מחדש של מקטעים שמעוותת את הגיאומטריה המוצגת
- חלון סיכום שמחשב ממוצע של ממוצעים במקום המדד הנכון
- השוואת ריצות שלא מיישרת נכון לפי אחוז מרחק

מה נבדק	כמה
רצועת הדשבורד – רצף, נפת, לוח שנה, ספירה לאחור (למרוץ, או לסוף תוכנית בלי מרוץ), תחת כל אזור זמן	45
גיאומטריית גרף האימון	27
דגימה מחדש של מקטעים	25
אזורי דופק וקצב	17
התוכנית האמיתית לתצוגה	18
השוואת מתוכנן מול בפועל	15
שיאים אישיים	14
בניית ההסבר	20
גרף זעיר	11

מה נבדק	כמה
ערכת הרכיבים המשותפת – גיאומטריית עמודות, מצבי תא יום, רשת חודש	41
חלון הסיכום והממוצעים המשוקללים	16
השוואת ריצות – יישור לפי אחוז, פיצול לרבעים, הכרעה לפי יעילות	26
הנוסחאות שהמוצר מציג – כיסוי, טווחים, מקורות	7
לוח המספרים – טווחי הקריאה, היסטוריה לפי מקור (תמונת מצב / ריצה / לילה), נפח שבועי כעמודות עם השבוע הרץ	15
השאלות של Ask Runi – כל שאלה, ומה היא עונה כשאינן מספיק נתונים	50
סיווג סוג האימון והחציון שהוא נמדד מולו	13

צד המאמן

מה יכול להשתבש:

- רוסטר שמציג מתאמן ששייך למאמן אחר
- מחזור שמקבץ רץ למרוץ או למרחק הלא נכון
- שיבוץ למחזור שנחסם בשקט על ידי מדיניות ההרשאות
- עריכת תבנית שמתעלמת מכך שהשבוע כבר ננעל

מה נבדק	כמה
רוסטר ודגלי תשומת לב	30
מחזורי הכנה – קיבוץ לפי מרוץ ומרחק, שבוע של כל רץ, וסינון	26
יומן – שנה / חודש / שבוע	21
תבניות ותקינות	15
שבוע של תבנית – מי נמצא בו, ומה כבר לא ניתן לשינוי	13

מה למדנו: תיקון ההטיה ב-ACWR ותיקון "מי קבע את האימון" הם שני מקומות שבהם התנהגות נכונה-כלפי-חוץ יכולה בכל זאת להטעות משתמש – הבדיקות שם לא בודקות רק "התקבל מספר", אלא שהמספר לא סותר את מה שהמשתמש כבר יודע (שהוא לא נגע בתוכנית, שהשבוע שלו לא השתנה סתם).

2. בדיקות לקלטים לא תקינים

סוגי הקלט שנבדקו: מספרים בטווח, תאריכים, ערכים מרשימה סגורה, קבצים ותבניות – לכל אחד נבדק גם המקרה שבו הוא מגיע תקין-כלפי-חוץ אך פסול (מרחק בלי תאריך, קצב שנכתב בפורמט זר).

- **סכימות 7 – Zod** בדיקות: מרחק מרשימה סגורה, תאריך בפורמט מלא, מספרים בטווח
- **מרוץ יעד** – מרחק בלי תאריך **נדחה**. קודם זה עבר בשקט וזרק חצי מהקלט

- **תכנית מאמן** – פאזות שלא מסתכמות באורך התוכנית; שבוע שאינו 7 ימים; שבוע בלי מנוחה או בלי ריצה ארוכה – כולן נדחות עם הסבר
- **קצב** – קלט לא תקין מחזיר "אין ערך", לא NaN
- **תמונה** – קובץ שאינו תמונה, וגודל מעל התקרה
- **תוכנית של הרץ עצמו** – אפס שבועות, שבוע בלי ריצה, מרחק 0 או 80 ק"מ, קצב שנכתב "5.30" – כולם נדחים עם משפט שאומר מה לתקן

3. בדיקות לתהליכים עסקיים

נבדקות ברמת הלוגיקה, לא בדפדפן:

תהליך	הבדיקה
ריצה נכנסת → מוכנות	סדרת ריצות → תמונות מצב יומיות תקינות
מרוץ יעד → תוכנית	תוכנית שנגמרת ביום המרוץ, בפאזות נכונות
עומס עולה → הקטנה	יחס מעל הסף → הקטנה עם סיבה
עומס יורד → החזרה	היחס חוזר → האימון חוזר למרחקו
מאמן עורך → המנוע מכבד	origin = coach → המנוע מדלג
ריצה נערכה בדיעבד	חותמת השינוי זזה → הניתוח מתבצע שוב
דופק חסר	נפילה לקצב במקום להעלים את האימון
רץ בלי ריצות → תוכנית	שני מספרים במקום היסטוריה → תוכנית בגודל שהוזן, קצבים מזמן היעד
רץ עוזב תוכנית	התוכנית נסגרת (abandoned), הריצות נשארות, ועמוד התוכנית חוזר לשלושת הנתיבים – נבדק ידנית

מה למדנו: הקטנת עומס וההחזרה ממנה נבדקות כזוג, לא רק לכיוון אחד – מדיניות שמקטינה עומס וטועה בדרך חזרה שקולה לתקלה שלא מטפלים בה. עזיבת תוכנית נבדקת ידנית כי היא נוגעת בשלוש טבלאות בבת אחת (תוכנית, ריצות, מחזור), ואוטומציה חלקית הייתה מסתירה בדיוק את המקרה שרוצים לתפוס.

4. בדיקות הרשאות

נבדקו ידנית מול המסד, לא אוטומטית – אין פרויקט בדיקות נפרד.

מה נבדק	תוצאה
מדיניות יצירת קשר מאמן-מתאמן	נמצאה פרצה . ראו סעיף 8 ומסמך האבטחה
אתלט מנסה להגיע ל- coach	חסום ב-layout אחד לכל הענף
מאמן עורך אימון של מתאמן	מותר, ונבדק שהכתיבה באמת קרתה
מאמן עורך אימון של מי שאינו שלו	נחסם

מה נבדק	תוצאה
עזיבת מאמן ע"י המתאמן	היה שבור – דיווח הצלחה והתאים אפס שורות
מאמן בונה תוכנית לרץ במחזור	מותר מאז מיגרציה 0020; נבדק בפרודקשן – הרץ רואה את התוכנית בשבוע הנכון
מאמן משבץ רץ למחזור	נמצא סירוב שקט (31/8). העדכון על שורת הקשר הוא של הרץ בלבד (0013); "נוסף" הופיע מעל מחזור ריק. תוקן ב-0021 בפונקציה צרה, והפעולה בודקת שהשורה השתנתה
מתאמן שישי מקליד קוד של מאמן בחבילת בסיס	join_coach מסרב עם "roster full" והמסך מסביר; מתאמן שכבר ברוסטר אינו נספר שוב – נבדק ידנית מול המסד

זה הלקח המרכזי בסעיף הזה: פוסטגרס לא זורק כשמדיניות מסרבת – הוא לא מתאים שורות והפעולה מדווחת הצלחה. בדיקה שרק בודקת "לא הייתה שגיאה" תעבור על קוד שבור. לכן כל כתיבה רגישה מבקשת בחזרה את השורות שהשתנו.

5. בדיקות למסד הנתונים

מה	איך אומת
ייחודיות ריצה לפי מקור ומזהה חיצוני	יבוא חוזר לא מכפיל
ייחודיות תמונת מצב לפי (משתמש, תאריך)	כתיבה חוזרת מעדכנת
מדיניות ההרשאות	שאילתה ישירה ל-pg_policy
קיום כל עמודה שנוספה	information_schema.columns
אילוץ origin	pg_constraint
האינדקסים	pg_indexes

כלל שנקבע אחרי שנכוונו: מיגרציה מאומתת מול information_schema ו-pg_policy – לא מול הודעת ההצלחה. מיגרציה דווחה כמוצלחת ולא רצה, והתגלה רק כשפיצ'ר שלם לא עבד.

6. בדיקות למקרי קצה

זמן ואזור זמן – 24 בדיקות

הבאג: התאריך חושב באזור הזמן של השרת. Vercel רץ ב-UTC, המשתמשים בישראל. בין חצות ל-03:00 המערכת חשבה שזה עוד אתמול – הרצף היה קצר ביום, והתוכנית תייגה את האימון של אתמול כ"היום".

וזה החלק המעניין: הבדיקה הקיימת הגדירה את אזור הזמן לפני שרצה, ולכן הוכיחה התנהגות שלייצור מעולם לא הייתה.

הבדיקות החדשות רצות תחת כל אזור זמן: UTC 22:30 שהוא כבר מחר בישראל, מעבר שנה, וצהריים שבו שניהם מסכימים.

גרפים – 63 בדיקות

נקודה אחת - אפס נקודות - כל הערכים זהים - טווח מנוון - ערכים שליליים - פערים בנתונים. כל אחד מהם מצויר נכון או מוסתר – **ולא הופך ל-NaN בתוך מסלול SVG**, שהדפדפן פשוט לא מצייר.

נתונים חסרים

משתמש חדש בלי ריצות - ריצה בלי דופק - מרוץ קרוב מדי (סירוב מנומק) - אין היסטוריה לבניית תוכנית - מאמן בלי מתאמנים.

7. בדיקות UI

לא אוטומטיות. במקומן: 4 בדיקות לראשי התיבות של תמונת הפרופיל, ו-15 לטקסטים ולחישובי מסך ההגדרות – הלוגיקה שמאחורי התצוגה, לא הרינדור.

וסבב ידני מתועד – סעיף 8.

8. בדיקות ידניות מתועדות

שלושה סבבים

שלושה סבבים ידניים, וכל אחד תפס סוג באג שהאחרים לא היו תופסים. התייעוד המלא בתיקיית docs/process בריפו.

סבב	מה נבדק	נמצאו ותוקנו
סריקת עומק - 19.8	חמישה תחומים: מחזור חיי האימות, בידוד בין משתמשים, טריות נתונים, קישוריות מקצה לקצה, גרפים	47; שישה חסמו שחרור, ובהם פרצת ההרשאה (מסמך אבטחת המידע, סעיף 4)
ביקורת חישוב - 25.8	מחלקת טעות אחת בכל הקוד: ממוצע של ממוצעים, קצב שממוצע כאילו אינו הופכי, תאריכים במילישניות	9; החשבון עבר לפונקציות טהורות, ולכל אחת בדיקה שנכשלת על החישוב הישן
סבב פרודקשן - 31.8	מעבר מלא באתר החי כמאמן וכרץ: מחזורים, תבניות, לוח המספרים, תוכנית עצמית, הגדרות, בית ריק	3; שיבוץ שסורב בשקט, בית שהציע לבנות תוכנית שכבר קיימת, טופס שהזיז תאריך מרוץ בשינוי שם

סבב ידני לפני הגשה

הרשמה • אישור מייל • התחברות ב-Enter • שחזור סיסמה • חיבור השעון וסנכרון • בחירת נתיב תוכנית (קוד / Runi עם תצוגה מקדימה / משלי) • פתיחת יום ובדיקה שהמספרים הם של המשתמש • לוח המספרים • עזיבת תוכנית • הרשמה כמאמן • בחירת חבילה בלשונית החיוב ומוקאפ אמצעי תשלום • מאמן ב- settings מופנה להגדרות המאמן • יצירת קוד מאמן • הצטרפות • מחזור: יצירה, שיבוץ, עריכה, סגירה • עריכת אימון • אתלט מנסה / coach (צריך 404) • כתובת לא קיימת (דף 404 של המוצר) • התנתקות.

מה למדנו: הסריקה תפסה שבירות מבניות, ביקורת החישוב תפסה טעות שיטתית שהתפזרה בכמה מסכים, וסבב הפרודקשן תפס גרסיות במעברים בין מסכים. אף סוג בדיקה אחד — לא אוטומטי ולא ידני — לא היה תופס את שלושתם לבד.

9. מגבלות ומה היה נבדק בהמשך

מגבלה	מה היה נדרש
אין בדיקות רינדור	React Testing Library לקומפוננטות המורכבות
אין E2E בדפדפן	Playwright לתהליכי ההרשמה, החיבור והתוכנית
הרשאות נבדקות ידנית	פרויקט Supabase לבדיקות ו-client לכל תפקיד
הדיבור עם intervals.icu לא ממוקה	שרת מדומה לתגובות 429 ו-5xx
אין מדידת כיסוי	<code>vitest --coverage</code> , וסף בשילוב CI
אין בדיקות עומס	k6 או Artillery מול הפריסה

המגבלה שהכי מטרידה אותי היא השלישית. ההרשאות הן החלק שבו טעות עולה הכי ביוקר — וזה בדיוק החלק שנבדק בעיניים. הפרצה שנמצאה מוכיחה גם שהבדיקה הידנית עבדה, וגם שהיא הגיעה מאוחר. והסירוב השקט של השיבוץ למחזור הוא אותו לקח בפעם השנייה: מדיניות השתנתה במיגרציה אחת, עמודה נוספה באחרת, ואף בדיקה לא רצה בין השתיים.

מסמך 5 – אבטחת מידע

תוכן עניינים

2.....	הרעיון המרכזי
2.....	1. אימות (Authentication).....
2.....	הסשן בעוגייה, לא ב-localStorage.....
3.....	PKCE – ולמה נדרשו שני נתיבים.....
3.....	מה תוקן בסריקה.....
3.....	2. הרשאה (Authorization).....
3.....	שלוש שכבות, ורק אחת מהן באמת מגנה.....
4.....	הדפוס.....
4.....	3. פעולות שדורשות משתמש מחובר.....
4.....	4. מניעת גישה לנתוני משתמש אחר.....
4.....	הפרצה שנמצאה – והסעיף החשוב במסמך.....
5.....	למה זה קרה.....
5.....	התיקון – מיגרציה 0013.....
5.....	אותו גבול, מהצד השני – מיגרציות 0020 ו-0021.....
6.....	וההרשאה עדיין מגיעה מהמסד, לא מהקוד.....
6.....	5. ולידציה של קלטים.....
6.....	הכלל.....
6.....	דוגמאות.....
6.....	שתי ולידציות שהן אבטחה ולא נוחות.....
6.....	6. הגנה על קריאות ל-API.....
7.....	7. שמירת סודות.....
7.....	שלושה כללים.....
8.....	8. מה עדיין פתוח –.....
8.....	סיכונים ידועים.....
9.....	מה שהייתי מוסיף ראשון.....
9.....	הלקח המרכזי.....

הרעיון המרכזי

ההרשאה נאכפת במסד, לא בקוד.

לכל טבלה צמודה מדיניות שאומרת מי רשאי לראות ולשנות כל שורה. הכלל נאכף בפוסטגרס בכל שאילתה – גם אם הקוד מבקש משהו אחר. באג באפליקציה יכול להציג את הדבר הלא נכון; הוא לא יכול להביא נתונים שהמשתמש לא רשאי לקבל.

44 מדיניותיות. Row Level Security מופעלת על כל 17 הטבלאות – בלי יוצא מן הכלל. ותשע פונקציות SECURITY DEFINER, כולן צרות: שני טריגרים של Supabase Auth (יצירת פרופיל וסנכרון מייל), עוזר-מדיניות אחד (is_coach_of), ושש של צד המאמן – קוד הצטרפות, הצטרפות, שיבוץ למחזור וריקונו. העיקריות מתועדות כאן.

והמסמך הזה מספר גם מה שלא עבד: מצאנו פרצה אמיתית, בדקנו למה היא נוצרה, תיקנו ואימתנו. סעיף 8 מפרט מה עדיין פתוח.

המפה: שמונה חזיתות, ומה כל אחת מבטיחה.

חזית	מה מובטח
1. אימות	מי אתה, לפי Supabase Auth – לא קוד שכתבנו
2. הרשאה	מה מותר לך, לפי מדיניות RLS על כל טבלה
3. פעולות מחוברות	אין נתון בלי חשבון – Middleware + בדיקה + RLS
4. בידוד בין משתמשים	הפרצה שנמצאה ותוקנה; הגבול האמיתי הוא מדיניות ה-INSERT
5. ולידציית קלטים	בדיקת דפדפן היא נוחות; ההגנה בשרת ובמסד
6. הגנה על API	getUser(), סוד קבוע-זמן, סכימת Zod קשיחה
7. שמירת סודות	מה חשוף ללקוח בכוונה, ומה לא נחשף לעולם
8. מה עדיין פתוח	סיכונים ידועים ומודעים, לא מוסתרים

1. אימות (Authentication)

הנושא: מי אתה, לפי Supabase Auth – לא קוד שכתבנו.

איום: אימות שנכתב מאפס הוא המקום הכי קל בעולם לטעות בו; קישור מייל בלי נתיב תקין להחלפה לסשן משאיר חשבון קיים ובלתי שמיש.

מענה: Supabase מנהל סיסמה, גיבוב ואימות מייל; הסשן בעוגיית httpOnly ולא ב-localStorage; שני נתיבי PKCE (callback + confirm) סוגרים את מסלול המייל; וחמישה ליקויים נוספים נמצאו ותוקנו בסריקה (טבלה בהמשך).

Supabase Auth, מייל וסיסמה. שמירת הסיסמה, גיבובה ואימות המייל הם באחריות Supabase – **לא כתבנו אימות בעצמנו**, וזו החלטה: אימות שנכתב מאפס הוא המקום הכי קל בעולם לטעות בו.

הסשן בעוגייה, לא ב-localStorage

`supabase/ssr` שומר את הסשן בעוגיית HTTP. שתי סיבות:

היא מגיעה לשרת. העמודים נבנים בשרת, וצריך לדעת שם מי המשתמש. **localStorage** נגיש ל-JavaScript. כל סקריפט בדף יכול לקרוא ממנו. עוגייה עם **httpOnly** – לא.

PKCE – ולמה נדרשו שני נתיבים

הקישור במייל מגיע עם **קוד חד-פעמי, לא עם סשן**. צריך להחליף אותו לסשן **בצד השרת**, כדי שהעוגייה תיכתב בתשובה שהדפדפן ישמור. לכן **auth/callback/** ו-**auth/confirm/**.

מה קרה בלי זה: ההרשמה עבדה, המשתמש נוצר, והקישור הוביל ל-404. חשבון קיים ובלתי שמיש. נמצא רק כשמישהו ניסה להירשם באמת.

מה תוקן בסריקה

הליקוי	למה זה נחשב
מסך שחזור סיסמה לא היה קיים	קישור השחזור לא הוביל לשום מקום
כתובת החזרה לא נשלחה בהרשמה	הקישור הלך להגדרה גלובלית אחת, משותפת ל-localhost ולייצור
התנתקות ניתקה בכל המכשירים	לחיצה בלפטופ משותף העיפה גם את הטלפון באמצע ריצה
הרשמה עם מייל קיים דיווחה הצלחה	Supabase מחזיר תשובה זהה בכוונה – אחרת הטופס הוא כלי לגלות למי יש חשבון. הסימן הוא רשימת זהויות ריקה
מינימום 8 תווים נאכף גם בהתחברות	כלל על בחירת סיסמה, לא על הקלדת סיסמה קיימת

תובנה: הליקוי המרכזי שנמצא בסריקה (מסך שחזור סיסמה שלא הוביל לשום מקום) לא עלה בקריאת קוד – הוא התגלה רק כשמישהו ניסה להשתמש בתכונה בפועל. סימן שחלק מהבאגים מתגלים רק בזרימה מלאה.

2. הרשאה (Authorization)

הנושא: מה מותר לך, לפי מדיניות RLS על כל טבלה – לא לפי מי שאתה מצד הקוד.

איום: שלוש שכבות בודקות הרשאה (Middleware, בדיקה בקוד, RLS), אבל שתיים מהן הן קוד – וקוד אפשר לשכוח. הגנה שנשענת על שכבה אחת שאפשר לפסוח עליה בטעות אינה הגנה אמיתית.

מענה: RLS היא השכבה שבאמת אוכפת, כי היא צמודה לטבלה בפוסטגרס ולא לנתיב קוד; המדיניות בודקת קשר אמיתי (coach_athletes), לא שדה role על המשתמש.

שלוש שכבות, ורק אחת מהן באמת מגנה

שכבה	מה עושה	למה לא מספיקה
Middleware	מפנה מבקר לא מחובר	ניתוב בלבד. לא נוגע בנתונים
בדיקה בפעולה	getUser() לפני כל גישה	קוד – וקוד שוכחים
RLS	הכלל צמוד לטבלה	–

הדפוס

אתלט רואה שורה אם `user_id = auth.uid()` **מאמן** רואה אותה אם קיים קשר פעיל ב-
`coach_athletes` – דרך פונקציה `is_coach_of()` שמסומנת `SECURITY DEFINER`, כי היא צריכה לקרוא
טבלה שהקורא עצמו לא רשאי לקרוא.

התפקיד אינו הרשאה. `role` מחליט מה מוצג. מה שמותר נקבע בקשר.

תובנה: התפקיד (role) אינו הרשאה. הוא מחליט מה מוצג למשתמש; מה שמותר בפועל נקבע רק לפי קשר
קיים בטבלה.

3. פעולות שדורשות משתמש מחובר

הנושא: אין נתון בלי חשבון, בשום נתיב.

איום: שכבת הגנה אחת שנשכחת (Middleware שלא מכסה נתיב, Server Action בלי בדיקת משתמש) חושפת
נתונים בלי שהקוד "נשבר" בשום מקום גלוי.

מענה: שלוש שכבות על כל פעולה (`getUser()` Middleware, בקוד, RLS במסד) – וכלל שנאכף בבנייה עצמה:
קובץ "use server" מייצא רק פונקציות אסינכרוניות, כי כל ייצוא הוא נקודת קצה ציבורית.

כולן. אין נתון במוצר שנגיש בלי חשבון.

- **Middleware** מגן על כל הנתיבים המוגנים
- **כל Server Action** בודקת `getUser()` לפני כל שאילתה
- **RLS** מסננת גם אם השתיים הקודמות נכשלו

וכלל שנאכף בקוד: קובץ "use server" מייצא רק פונקציות אסינכרוניות, כי כל ייצוא הוא נקודת קצה
ציבורית. ייצוא של קבוע הפיל את הבנייה פעמיים – וזו התנהגות נכונה.

תובנה: ייצוא של קבוע (לא פונקציה) מקובץ כזה הפיל את הבנייה בפועל – וזו התנהגות רצויה: מוטב שהבנייה
תיכשל מאשר ששכבת ההגנה הזו תישכח בשקט.

4. מניעת גישה לנתוני משתמש אחר

הפרצה שנמצאה – והסעיף החשוב במסמך

הנושא: הפרצה שנמצאה ותוקנה: הגבול האמיתי בין משתמשים הוא מדיניות ה-INSERT על `coach_athletes`.

איום: מדיניות ה-INSERT בדקה את העמודה הלא נכונה (`coach_id` במקום `athlete_id`) – כל מי שידע להריץ
שורת קוד אחת בקונסול הדפדפן יכול היה להכריז על עצמו כמאמן של כל משתמש ולקרוא את נתוניו.

מענה: נמצא, אומת מול `pg_policy`, ותוקן במיגרציה 0013 באותו יום. הפירוט המלא בהמשך הסעיף.

מיגרציה 0001 יצרה את המדיניות הזו:

```
create policy ca_insert on public.coach_athletes for insert  
;with check (coach_id = auth.uid())
```

התנאי היחיד הוא שאתה המאמן. שום דבר לא מגביל את `athlete_id`.

לקוח Supabase נשלח לדפדפן עם הסשן של המשתמש, ולכן שורה אחת בקונסול הספיקה:

```
supabase.from('coach_athletes')
insert.({ coach_id: <אני>, athlete_id: <כל אחד>, status: 'active' })
```

מרגע זה `is_coach_of()` מחזירה אמת, וכל מדיניות שנשענת עליה נפתחת: הפרופיל של הקורבן – מייל, גיל, מין, גובה, משקל – הפעילויות, היסטוריית המוכנות, מרוץ היעד והתוכנית. ומיגרציה 0009 הוסיפה למאמן הרשאת כתיבה על אימונים, אז גם אפשר היה לשכתב לו את התוכנית. הקורבן לא מקבל שום התראה.

למה זה קרה

כל הרשאה בצד המאמן מסתכמת בשאלה "יש שורה ב-coach_athletes?" – מה שהופך את מדיניות ה-INSERT על הטבלה הזו לכל גבול האבטחה. והיא בדקה את העמודה הלא נכונה.

זו טעות של תו אחד במקום שהוא היחיד שחשוב.

התיקון – מיגרציה 0013

```
create policy ca_insert on public.coach_athletes for insert
with check (athlete_id = auth.uid())
```

רק המתאמן יוצר את הקשר. וזה כבר איך שהמוצר עובד: המאמן מייצר קוד, המתאמן מקליד – והקלדת הקוד היא ההסכמה. המדיניות רק סוגרת דרך שנייה, לא-מוסכמת, פנימה.

ותוקן גם: מדיניות המחיקה הרשתה רק למאמן – אז מתאמן לא יכול היה לעזוב. הכפתור דיווח הצלחה והתאים אפס שורות. עכשיו שני הצדדים רשאים, ואתלט לעולם לא צריך רשות כדי להפסיק להיות נצפה.

אומת מול pg_policy אחרי ההרצה – לא מול הודעת ההצלחה.

אותו גבול, מהצד השני – מיגרציות 0020 ו-0021

המחזורים של המאמן הוסיפו ל-coach_athletes עמודה, cycle_id – המאמן הוא שמשבץ רץ למחזור. אבל 0013 סגרה את העדכון על השורה הזו לרץ בלבד, ובצדק. פוסטגרס סירב בשקט: "נוסף" הופיע על המסך, והמחזור נשאר ריק.

הפיתוי היה לפתוח UPDATE למאמן. זה היה מחזיר את הפרצה מסעיף זה בדלת האחורית – מאמן שיכול לעדכן את השורה יכול גם להפוך status של רץ שמעולם לא הקליד את הקוד שלו ל-active. מדיניות שורה לא יודעת להגיד "העמודה הזו כן, האחרות לא".

ולכן מיגרציה 0021 מוסיפה פונקציה אחת – set_athlete_cycle – שרצה כ-SECURITY DEFINER, בודקת שהקורא הוא מאמן פעיל של הרץ ובעל המחזור, כותבת ל-cycle_id בלבד, ומחזירה אם שורה השתנתה. status נשאר בדיוק כפי ש-0013 השאירה אותו. באותה רוח, 0020 נתנה למאמן INSERT ו-UPDATE על goal_races – training_plans של רציו בלבד, דרך is_coach_of – כדי שיוכל לבנות להם תוכנית – ולא נגעה בשורת הקשר.

העיקרון שנשאר: הרחבת הרשאה נעשית בפונקציה צרה עם בדיקות משלה, לא במדיניות רחבה. וכל כתיבה כזו מדווחת אם שינתה שורה – כי סירוב שקט הוא הבאג שמצאנו פעמיים.

ואותו כלל בכיוון ההפוך – מכסת המושבים (0022). מתאמן שמקליד קוד של מאמן אינו רשאי לקרוא את המנוי של אותו מאמן (subscriptions היא עצמית בלבד). לכן הבדיקה "יש עוד מושב?" לא יכולה להיעשות בקוד הלקוח או במדיניות – היא יושבת בתוך join_coach, שממילא רצה כ-definer, וסופרת מול המכסה בלי לחשוף אותה. בחירת החבילה עצמה היא כתיבה של המאמן על השורה שלו בלבד, ופרימיום נפתח היום בלי חיוב – הדף אומר זאת, כדי שלא יהיה מסך שנראה כמו תשלום ואינו כזה.

והרשאה עדיין מגיעה מהמסד, לא מהקוד

בזמן שהפרצה הייתה פתוחה, שאר המדיניות עשו את שלהן: אתלט לא יכול היה לקרוא אתלט אחר. מה שנפרץ היה **הכניסה לרשימת המורשים** – ולא הרשימה עצמה.

5. ולידציה של קלטים

הנושא: מה שהדפדפן מאשר הוא נוחות; ההגנה האמיתית היא בשרת ובמסד.

איום: בדיקה שקיימת רק בדפדפן נמצאת אצל התוקף וניתנת לעקיפה לגמרי – כל קלט שלא נבדק שוב בשרת חשוף.

מענה: כל קלט עובר שלוש שכבות (דפדפן למשוב, Server Action עם Zod ובדיקת בעלות, מסד עם check/unique/RLS); ושתי בדיקות מיוחדות (כתובת הפניה יחסית בלבד, זהות הבעלים) מסווגות כאבטחה ולא נוחות.

הכלל

בדיקה בדפדפן היא נוחות, לא הגנה – היא נמצאת אצל התוקף וניתנת לעקיפה. כל קלט נבדק **שוב** בשרת.

שכבה	מה
דפדפן	משוב מיידי
Server Action	ההגנה . Zod, טווחים, ובדיקת בעלות
מסד	check, unique, מפתחות זרים, RLS

דוגמאות

גיל 10-100 • מרחק מרוץ מרשימה סגורה • תאריך בפורמט מלא • מרחק אימון 0-100 ק"מ • תמונה: סוג וגודל • תבנית מאמן: פאזות שמסתכמות נכון ושבע בן 7 ימים.

שתי ולידציות שהן אבטחה ולא נוחות

כתובת הפניה – רק נתיב יחסי. כתובת מלאה הופכת את מסך ההתחברות שלנו **להפניה פתוחה** לאתר של תוקף.

זהות הבעלים – פעולה שמקבלת מזהה מהלקוח בודקת שהמזהה שייך למי שמבקש. נמצא ותוקן: ניתוח פעילות שאב את הספים והתוכנית **של הצופה** במקום של בעל הריצה – מאמן שפתח ריצה של מתאמן ראה אזורי דופק שגויים.

תובנה: הבאג שנמצא בזהות הבעלים (ניתוח פעילות שאב נתונים של בעל הריצה האמיתי, לא הצופה) מראה שבדיקת "יש הרשאה לטבלה" לא מספיקה – צריך לבדוק גם שהשורה הספציפית שייכת למי שמבקש אותה.

6. הגנה על קריאות ל-API

הנושא: כל נקודת קצה מוגנת בדרך המתאימה לה – משתמש מחובר, או סוד משותף.

איום: נקודות קצה שמשרתות לא-משתמשים (cron, webhook) לא יכולות להסתמך על `getUser()`; השוואת סוד רגילה (`===`) מדליפה מידע על כמה תווים נכונים, לפי זמן התגובה.

מענה: Server Actions מוגנות ב-`getUser()+RLS`; נקודות ה-`cron/webhook` מוגנות בסוד בכותרת; וההשוואה היא `timingSafeEqual` (קבועת-זמן), לא `===`, כדי לא להדליף מידע בזמן התגובה.

נקודה	הגנה
Server Actions	<code>getUser() + RLS</code>
<code>api/cron/sync-intervals/</code>	סוד בכותרת, בהשוואה קבועת-זמן
<code>api/webhooks/health/</code>	סוד + סכימת Zod קשיחה
<code>auth/callback, /auth/confirm/</code>	קוד חד-פעמי; <code>next</code> נבדק כנתיב יחסי

למה השוואה קבועת-זמן. `a === b` נשברת בתו הראשון שנבדל, וזמן התגובה מדליף כמה תווים התאימו. `timingSafeEqual` משווה תמיד את כל האורך.

תובנה: ההבדל בין `===` ל-`timingSafeEqual` הוא מהסוג שקל לפספס בביקורת קוד רגילה — הוא לא נראה כמו בעיית אבטחה, הוא נראה כמו עניין של סטייל כתיבה.

7. שמירת סודות

הנושא: מה חשוף ללקוח בכוונה, ומה לא נחשף לעולם.

איום: מפתח שמגיע לדפדפן בטעות (במקום להישאר בשרת) עוקף כל הגנה אחרת שבנינו — `RLS` לא עוזר אם המפתח שעוקף אותו יושב בדפדפן של מישו.

מענה: ארבעה סוגי סוד מסווגים בבירור לפי חשיפה (טבלה בהמשך); `env.local` לא נכנס לגיטימי; מפתח המשתמש רץ רק בשרת ומוצג בחלקיות בהגדרות; ומפתח ה-`service-role` (שעוקף `RLS`) רק במשימה היומית, בלי משתמש מחובר מאחוריו.

סוד	איפה	נחשף ללקוח?
<code>NEXT_PUBLIC_SUPABASE_URL / ANON_KEY</code>	Vercel	כן, בכוונה — מוגבל ע"י <code>RLS</code>
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Vercel, שרת בלבד	לא. עוקף <code>RLS</code> לגמרי
<code>CRON_SECRET, HEALTH_WEBHOOK_SECRET</code>	Vercel, שרת בלבד	לא
מפתח <code>intervals.icu</code> של המשתמש	טבלה, מאחורי <code>RLS</code>	לא — ולעולם לא

שלושה כללים

- `env.local` לא נכנס לגיטימי. `gitignore` ו-`env`, ו-`env.example` מתעד את השמות בלי הערכים.
- המפתח האישי לא מגיע לדפדפן. הסנכרון רץ רק בשרת. הפאנל בהגדרות מציג ארבעה תווים אחרונים בלבד.

- **מפתח ה-service-role עוקף את כל ההרשאות** – ולכן הוא רק במשימה היומית, שאין לה משתמש מחובר לפעול בשמו.

תובנה: מפתח ה-service-role הוא הסוד החשוב ביותר במוצר בדיוק כי הוא עוקף את כל שאר ההגנות שתוארו בסעיפים הקודמים – ולכן הוא מוגבל לתהליך אחד ויחיד, בלי משתמש מחובר שיכול לכוון אותו.

8. מה עדיין פתוח –

הנושא: הסיכונים שעדיין פתוחים, מודעים ולא מוסתרים.

איום: שש חסרים ידועים: אין הגבלת קצב, אין בדיקה אוטומטית שכל Server Action בודקת אימות, תמונת פרופיל מנפחת שורה, אין יומן ביקורת, מיילי האימות יוצאים מדומיין בדיקה שמוסר רק לכתובת אחת, ומפתח intervals.icu לא מוצפן בשכבת היישום (פירוט בהמשך).

מענה: שלושת הראשונים בעדיפות (הגבלת קצב, בדיקה אוטומטית לאימות, יומן ביקורת) מפורטים למטה; השאר מתועדים ומחכים להחלטת תעדוף.

סיכונים ידועים

אין הגבלת קצב. אף נקודת קצה לא מוגבלת בקצב. אפשר לנסות סיסמאות בהתחברות, או להעמיס פעולות שרת. **מה נדרש:** הגבלה לפי IP וחשבון.

כל Server Action היא נקודת קצה ציבורית. כל אחת בודקת אימות, אבל **הגבול נשען על כך שאף אחת לא תישכח.** ויש כמה שנשארו מיוצאות בלי שאף מסך קורא להן. **מה נדרש:** בדיקה אוטומטית שכל ייצוא כזה מכיל בדיקת אימות.

תמונת הפרופיל נשמרת בתוך שורת הפרופיל כטקסט מקודד, עד ~280 אלף תווים. תקין, אבל מנפח שורה שנקראת בכל טעינה. **מה נדרש:** Supabase Storage.

יש רישום שגיאות, אין יומן ביקורת. כישלון בפעולת שרת נרשם ביומן של Vercel. אבל אין רישום של מי עשה מה – מי הצטרף למאמן, מי ערך אימון, מי שינה פרופיל. **מה נדרש:** יומן ביקורת לפעולות רגישות.

אימות המייל תלוי בדומיין שאין לנו עדיין. Confirm email דולק ו-Resend מחובר כ-SMTP, אבל בלי דומיין מאומת Resend שולח רק לכתובת בעלת החשבון – כל כתובת אחרת לא תקבל את מייל האימות. לבודקים יש חשבונות דמו שאינם צריכים אימות. **מה נדרש:** דומיין משלנו מאומת ב-Resend לפני פתיחה לקהל.

אין אימות דו-שלבי.

מפתח ה-intervals.icu נשמר כטקסט, מוגן ב-RLS. **מה נדרש:** הצפנה בשכבת היישום, כך שהמפתח לא נמצא באותו מקום כמו הנתונים.

מה שהייתי מוסיף ראשון

1. **הגבלת קצב על ההתחברות** – הכי זול, ומכסה את הווקטור הכי סביר.
2. **בדיקה אוטומטית שכל Server Action בודקת אימות** – הופכת כלל תרבותי לכלל אכיף.
3. **יומן ביקורת** – כשהיו משתמשים אמיתיים, השאלה "מי ראה מה" תישאל.

הלקח המרכזי

הפרצה הייתה **שורה אחת שבדקה את העמודה הלא נכונה**, במקום היחיד שבו כל ההרשאה מתרכזת. מה שהופך את זה ללקח ולא לתקלה: היא לא נמצאה בקריאת קוד ולא בבדיקות. היא נמצאה כששאלנו "מה בעצם מחליט שמישהו הוא מאמן שלי?" – ואז הלכנו לקרוא את המדיניות עצמה.

וזה הכלל שנשאר: מאמתים מול pg_policy ו-information_schema, לא מול הודעת ההצלחה. ומרחיבים הרשאה בפונקציה צרה, לא במדיניות רחבה – הלקח של 0021, שנלמד על אותה טבלה בדיוק.

מסמך 6 – סקייל וביצועים

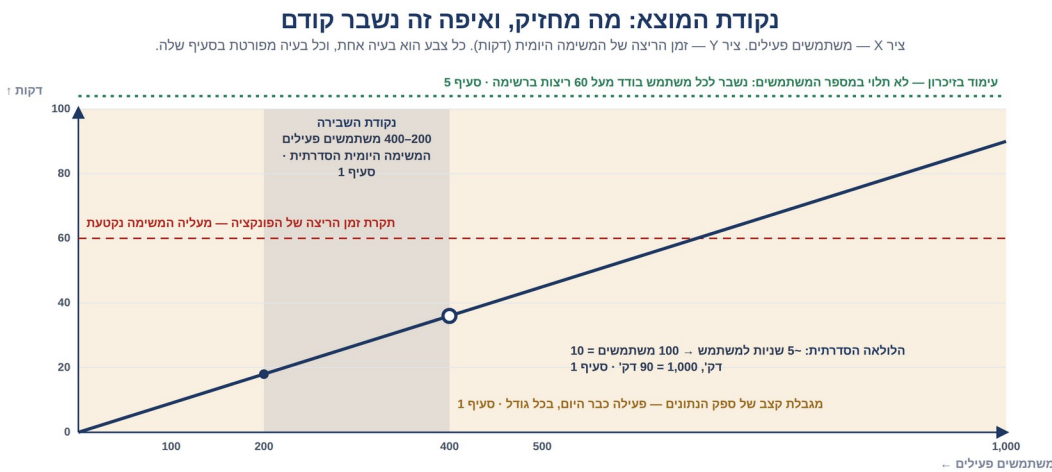
תוכן עניינים

2.....	נקודת המוצא.....
2.....	1. מה יקרה עם עשרות או מאות משתמשים.....
2.....	מה מחזיק בלי מאמץ.....
2.....	מה שנשבר ראשון – המשימה היומית.....
3.....	2. אילו שאילתות עלולות להיות כבדות.....
4.....	הדבר שהכי חשוב שנעשה נכון: אין N+1.....
4.....	3. אינדקסים.....
5.....	4. איך נמנעים מטעינה מיותרת.....
5.....	5. Pagination – ואיפה הוא חסר.....
6.....	ומה עוד לא מלא.....
6.....	6. הפרדה בין צד לקוח לצד שרת.....
7.....	7. מגבלות המוצר הנוכחי.....
8.....	8. מה היינו משפרים לגרסה הבאה.....
8.....	לפי סדר החזר ההשקעה.....

נקודת המוצא

המוצר בנוי לעשרות עד מאות משתמשים. המסמך אומר **איפה זה מחזיק, איפה זה נשבר, ומה נדרש כדי לעבור את הנקודה הזו** – כולל דבר אחד שכבר עכשיו הוא צוואר בקבוק ידוע.

סדרי גודל אמיתיים: משתמש פעיל מייצר ~5 ריצות בשבוע – כ-260 בשנה. חשבון הבדיקה מחזיק 130 ריצות ו-329 לילות. אלף משתמשים = כרבע מיליון ריצות. זה לא מספר גדול לפוסטגרס.



תקרת הפלטפורמה Vercel Hobby: זמן ריצה לפונקציה ו-cron פעם ביום. תקרה קשיחה. → סעיף 7	עימוד ביזכרון 60 ריצות לרשימה, לכל משתמש בנפרד. הפתרון: עימוד במסד. → סעיף 5	מגבלת קצב חיצונית intervals.icu מגביל קצב: לא תלוי בגדילה. הפתרון: אותו תור, בקצב מבוקר. → סעיף 1	המשימה היומית לולאה סדרתית על כל החיבורים. נשברת ראשונה, ב-200-400 משתמשים. הפתרון: תור עבודה. → סעיף 1
--	--	---	---

נושא	נקודת השבירה	מפורט בסעיף
המשימה היומית (לולאה סדרתית)	~200-400 משתמשים פעילים	סעיף 1
מגבלת קצב בצד ספקי נתונים חיצוניים	פעילה כבר היום, לא תלויה בגדילה	סעיף 1
עימוד ביזכרון	מעל 60 ריצות ברשימה לכל משתמש	סעיף 5
Vercel Hobby ב-תוכנית	תקרת פלטפורמה קשיחה	סעיף 7

1. מה יקרה עם עשרות או מאות משתמשים

מה מחזיק בלי מאמץ

קריאת מסך. כל עמוד נבנה בשרת עם מספר קבוע של שאילתות, כולן על אינדקס וכולן מסוננות למשתמש אחד. עוד משתמשים = עוד בקשות מקבילות, לא שאילתות כבדות יותר. Vercel מרחיב אופקית לבד.

החישובים. כל מודל העומס הוא פונקציות טהורות על מערכים באורך מאות. עלות זניחה, ואינה גדלה עם מספר המשתמשים.

מה שנשבר ראשון – המשימה היומית

```
} for (const conn of connections ?? [])
```

```
// רשת: ריצות + מקטעים
// חישוב
// כתיבה
importFromIcu(...);
await recomputeForUser(...);
await runPlanAdjustment(...);
{
```

לולאה סדרתית. משתמש אחד = 3-8 שניות, רובן המתנה לרשת.

משתמשים	זמן ריצה משוער
10	1~ דקה
100	10~ דקות
300	30~ דקות
1,000	90~ דקות – חורג ממגבלת הזמן

נקודת השבירה היא סביב 200-400 משתמשים פעילים, ומה שנגמר קודם הוא **זמן הריצה של הפונקציה,** לא המסד.

ויש מגבלה שנייה מאחוריה: intervals.icu מגביל קצב. אין למוצר היום אינטגרציה ישירה עם Garmin, Apple HealthKit או Suunto – הכל עובר דרך intervals.icu בלבד – כך שהתקרות שלהם לא רלוונטיות כרגע; אם תתווסף אינטגרציה ישירה בעתיד, שתיהן דורשות הרשמת-שותף ואישור מראש (Suunto אוסר מפתחים עצמאיים לגמרי), ולא מפרסמות מספרי קצב.

מה נדרש: תור עבודה – המשימה היומית רק מכניסה משימות, ועובדים מושכים מהתור בקצב מבוקר, עם ניסיון חוזר. מפרק גם את מגבלת הזמן וגם את מגבלת הקצב.

2. אילו שאילות עלולות להיות כבדות

השאילות הכבדות בפוטנציה הן אלה שסורקות היסטוריה שלמה או נתונים של הרבה משתמשים בבת אחת – רוסטר מאמן, סדרות זמן ארוכות, וסריקות שיאים. הטבלה הבאה מפרטת לכל אחת למה היא כבדה ומה נעשה כדי לרכך אותה.

שאילתה	למה כבדה	מה נעשה
רוסטר המאמן	נתונים על N מתאמנים	שאילות מבוססות-קבוצה – ראו למטה
סדרת המוכנות	84 ימים	אינדקס על (משתמש, תאריך); רק העמודות הנחוצות
שיאים אישיים	סריקת best_efforts	תקרה של 400 ריצות
יומן המאמן	כל האימונים של כל הרוסטר לשנה	חלון תאריכים + אינדקס
ניתוח מקטעים	הורדה מרשת לכל ריצה	קבוצות של 25, במקביליות מוגבלת ל-4
לוח המספרים	היסטוריה של שנה: תמונות מצב, עומס לכל ריצה, לילות	חלון שנה, 100 תמונות מצב, וההיסטוריה נבנית בזיכרון פעם אחת לטעינה

שאלתה	למה כבדה	מה נעשה
מחזורי המאמן	חברים × תוכניות × שבועות	שאלתה אחת לכל סוג עם in(ids); השבוע של כל רץ מחושב בזיכרון

הדבר שהכי חשוב שנעשה נכון: אין N+1

הדפוס הגרוע: לולאה על מתאמנים, ולכל אחד שאלתה. מאמן עם 30 מתאמנים = 30 הלוך-חזור לכל טעינת מסך.

מה שנעשה: שאלתה אחת לכל סוג נתון, עם in(ids), והצירוף בזיכרון. מאמן עם 30 מתאמנים עולה כמו **מאמן עם 3**.

זו ההחלטה היחידה שהכי משפיעה על התנהגות המוצר בגדילה, והיא נלקחה מראש.

בפועל: מאמן עם 30 מתאמנים טוען בכ-4 שאלות קבוצתיות בלבד (אחת לכל סוג נתון: רוסטר, מוכנות, שיאים, יומן) ולא ב-30 שאלות נפרדות – זה מה שעומד מאחורי "עולה כמו מאמן עם 3".

3. אינדקסים

18 אינדקסים נוצרו במפורש, ועוד מפתחות ראשיים ואילוטי ייחודיות שפוסטגרס ממש אותם כאינדקס גם הם. אף אחד מהם אינו "ליתר ביטחון": כל אינדקס שנוצר במפורש מכסה שאלתה שקיימת בקוד.

אינדקס	משרת	למה נבחר
activities (user_id, started_at)	רשימת ריצות, דשבורד, יכולת	כל שאלתה מסוננת למשתמש אחד וממוינת/מסוננת לפי תאריך – בלי זה כל שליפה היתה סריקה מלאה.
activities (user_id, source, external_id)	ייחודי – ייבוא חוזר לא מכפיל	מזהה ייבוא כפול במהירות ומונע אותו במבנה, לא רק בלוגיקה.
activities_derivation_idx	"מה עוד לא נותח" – חלקי, רק intervals.icu	מסנן רק שורות שטרם נותחו, כך שסריקת "מה עוד לא נותח" לא עוברת על כל הטבלה.
readiness_snapshots (user_id, date)	סדרת הדשבורד	הסדרה נשלפת לפי משתמש וחלון תאריכים, ממוינת.
plan_workouts (plan_id, day_date)	שבוע התוכנית והיומן	שבוע התוכנית והיומן נשלפים לפי תוכנית וטווח תאריכים.
plan_workouts_adjustable_idx	חלקי – רק origin = "generated"	מסנן רק אימונים שנוצרו אוטומטית – אלה שהמונע רשאי לגעת בהם, וזו השאלתה שרצה כל לילה.
coach_athletes – לפי מאמן ולפי מתאמן	שני כיווני החיפוש	החיפוש קורה משני הכיוונים – מאמן למתאמנים, ומתאמן למאמנים – וכל כיוון צריך אינדקס משלו.
coach_reminders (coach_id, done, due_date)	תזכורות פתוחות	מסנן תזכורות פתוחות (done=false) ומיין לפי תאריך יעד.

רשימת המחזורים של מאמן, ממוינת לפי יום מרוץ.	המחזורים של מאמן, ממוינים לפי יום המרוץ	coach_cycles (coach_id, race_date)
--	---	------------------------------------

שני אינדקסים חלקיים. אינדקס חלקי מכסה רק שורות שעונות לתנאי – קטן יותר, מהיר יותר, וזול יותר בכתיבה. `plan_workouts_adjustable_idx` מכסה רק את מה שהמנוע רשאי לגעת בו, וזו בדיוק השאילתה שרצה כל לילה.

האינדקס הכי חשוב הוא הייחודי על (user_id, source, external_id) – הוא לא רק מאיץ, הוא מונע כפילות מבנית. סנכרון חוזר מעדכן במקום להוסיף.

בניגוד לשני האינדקסים החלקיים שמייעלים שאילתות ספציפיות, האינדקס הזה קיים כדי למנוע מצב לא-תקין מבנית – כפל רשומות – ולכן הוא היחיד שבלעדיו קיים סיכון לנתונים שגויים, לא רק לביצועים.

4. איך נמנעים מטעינה מיותרת

• שולפים רק עמודות שצריך: כל שאילתה בקוד מבקשת את העמודות הנחוצות בלבד, כך שלא נשלפת כמות מידע עודפת בכל טעינה – משמעותי במיוחד בטבלת activities, שיש בה עמודות JSON גדולות (תוצרי ניתוח המקטעים), ובשורת ה-profiles, שיש בה את תמונת הפרופיל כ-data URL.

• המקטעים הגולמיים לא נשמרים: עשרות מגה-בייט לשנה למשתמש נשמרות כארבע עמודות נגזרות בלבד, והזרם הגולמי נמחק. אם אי-פעם יידרש נתון שכבר נגזר, אין צורך לשחזר אותו מגיבוי – הוא נשלף מחדש מ-intervals.icu ומחושב מחדש, כי הנתון הגולמי עצמו לא משתנה. זה חיסכון של סדר גודל בגודל המסד ובכל שאילתה.

• חלונות זמן. 84 ימי מוכנות בבית, 400 יום ייבוא, שנה ביומן ובלוח המספרים, 60 ריצות ברשימה.

• ההסבר נשמר, לא מחושב מחדש. נכתב פעם ביום ונקרא מהשורה.

• ומטמון שהיה טעות: בקשת נתוני ההתאוששות (readiness) נשמרה במטמון (cache – עותק זמני של תוצאה, כדי לא לחשב אותה מחדש בכל בקשה) לפי מפתח שנשאר קבוע לכל היום, אך עם תוקף (TTL) של שעה בלבד. התוצאה: "סנכרן עכשיו" לא יצר מפתח חדש – הוא פגע באותו מפתח הישן וקיבל בחזרה תשובה ממועד קודם. הוסר. הלכח: מטמון על נתיב שהמשתמש מפעיל באופן יזום ומצפה לתוצאה טרייה הוא באג, לא אופטימיזציה.

5. Pagination – ואיפה הוא חסר

הבהרה: העימוד כאן הוא עימוד בממשק המוצר – כמה שורות מוצגות למשתמש בכל פעם – ולא עימוד על מסד הנתונים עצמו. זה האחרון הוא בדיוק מה שחסר, ומפורט בתת-הסעיף הבא.

תקרה	מוצג איפה	הקשר
60 ריצות	רשימת הפעילויות (לכל משתמש בנפרד)	כ-3 חודשים לרץ טיפוסי
84 ימים	גרף סדרת המוכנות (לכל משתמש)	חלון התצוגה היומי של ציון המוכנות
400 ריצות	סריקת שיאים אישיים – PR (ההיסטוריה של אותו משתמש בלבד, לא של כל המסד)	מבטיח שהסריקה לא עוברת על כל הטבלה

שנה (365 ימים)	יומן המאמן ולוח המספרים (לכל משתמש)	חלון ברירת המחדל להיסטוריה מוצגת
----------------	-------------------------------------	----------------------------------

זה מספיק לרץ טיפוס – 60 ריצות הן כשלושה חודשים.

ומה עוד לא מלא

רשימת הפעילויות מעומדת – חמישה-עשר בעמוד, עם ניווט קדימה ואחורה. העימוד הוא בזיכרון, מעל אותה תקרה של 60 שורות: רץ עם שלוש שנים עדיין רואה את 60 האחרונות בלבד, אך יכול לדפדף ביניהן במקום לגלול רשימה אחת ארוכה. השלב הבא הוא להעביר את העימוד למסד.

מה נדרש: עימוד מבוסס-סמן (started_at < last) ולא offset – עם offset עמוד 40 מחייב את המסד לסרוק ולזרוק 2,400 שורות.

וגם בצד המאמן: לרוסטר, לתזכורות ולמחזורים של מאמן אין כרגע תקרת עימוד כלל – הם נטענים בשלמותם בכל טעינה. זה עדיין זניח כי מאמן טיפוס מנהל עשרות מתאמנים ולא מאות, אבל ברגע שרוסטר יגדל מעבר לכך, זה המקום הבא שידרוש עימוד.

6. הפרדה בין צד לקוח לצד שרת

למה מפרידים: רק לשרת יש גישה למסד הנתונים ולמפתחות האישיים (כגון מפתח ה-API של intervals.icu) – אלה לא יכולים להיחשף לדפדפן בלי לאבד כל הגנה. הלקוח מקבל תוצר מוכן: HTML ומספרים, לא גישה לנתונים הגולמיים.



איך זה מיושם: כל קוד שנוגע במסד או בסוד מסומן "use server" ורץ רק בשרת (בתיקיית src/actions) — קוד שהדפדפן אפילו לא מקבל. קומפוננטות React בצד הלקוח (טאבים, טפסים, ציור הגרפים) קוראות לפונקציות האלה ולא יודעות להתחבר למסד בעצמן. הטבלה הבאה ממפה מה קורה בכל צד — וזו בעצם התרשים שהתבקש.

רץ בשרת	רץ בלקוח
שליפה מהמסד	קליקים, טאבים, טפסים
כל מודל העומס	ציור SVG מנתונים מוכנים
דיבור עם ספקי הנתונים החיצוניים (כגון intervals.icu)	—

רץ בשרת	רץ בלקוח
ההסבר	—

הדפדפן לא מדבר עם המסד ישירות. אין שאילתה בקוד הלקוח.

שלוש תוצאות:

- ביצועים — החישוב קורה פעם אחת בשרת ולא בכל מכשיר. בלי ההפרדה, כל מכשיר היה צריך לחשב מחדש את כל מודל העומס (ACWR, מוכנות, קצבים) על ההיסטוריה המלאה של המשתמש — כ-260 ריצות בשנה — בכל טעינה, כולל על מכשירים חלשים יותר.
 - אבטחה — המפתח האישי ל-intervals.icu לא מגיע לדפדפן בשום שלב.
 - משקל — מה שנשלח הוא HTML ומספרים מוכנים, לא ספריית חישוב שהדפדפן צריך להוריד ולהפעיל.
- וההחלטה לוותר על ספריית גרפים** תומכת בזה: הגרפים כתובים ידנית ב-SVG, ושוקלים יחד פחות מהספרייה שהייתה מיובאת בשבילם.

7. מגבלות המוצר הנוכחי

#	המגבלה	נשבר כ-
1	המשימה היומית סדרתית	~200-400 משתמשים פעילים
2	מגבלת קצב בצד ספקי הנתונים החיצוניים (intervals.icu ודומים)	ריבוי משתמשים במקביל
3	העימוד בזיכרון, מעל תקרה של 60 שורות	רץ עם היסטוריה ארוכה
4	תמונה בתוך שורת הפרופיל	מנפח שורה שנקראת בכל טעינה
5	אין הגבלת קצב	ניסיון זדוני, לא גדילה טבעית
6	חישוב מוכנות על 120 יום בכל סנכרון	מיותר כשהשתנה רק היום האחרון
7	תוכנית Hobby ב-Vercel	מגבלת זמן ריצה לפונקציה בודדת, ותקרת חריצי cron — cron הוא משימה שרצה אוטומטית בזמנים קבועים, כמו הסנכרון היומי.
8	סנכרון עד 24 שעות עיכוב	מוצרי, לא טכני — Vercel מתיר עד 100 חריצי cron לפרויקט (המוצר משתמש ב-1); אפשר לעבור לסנכרון פעמיים ביום בלי עלות נוספת
9	לוח המספרים בונה שנה של היסטוריה בכל טעינה	רץ עם שנים של נתונים — נדרש סיכום שבועי שמור

8. מה היינו משפרים לגרסה הבאה

לפי סדר החזר ההשקעה

1. תור עבודה במקום לולאה. כיום פונקציית ה-cron היחידה עוברת על כל המשתמשים ברצף, בתוך הרצה אחת שחוסמת גם במגבלת הזמן וגם במגבלת הקצב של intervals.icu. השיפור: ה-cron רק מכניס משימה לכל משתמש לתור; עובד אחד או יותר מושך משימה בכל פעם בקצב מבוקר, עם ניסיון חזור על כשל נקודתי בלי להפיל את כל הריצה – כך שמגבלת הזמן ומגבלת הקצב מתפרקות בבת אחת.
2. חישוב מצטבר במקום מלא. היום כל סנכרון מחשב מחדש את 120 הימים האחרונים של המוכנות, גם כשנוספה רק ריצה אחת. השיפור: לשמור את נקודת החיתוך האחרונה שחושבה ולחשב מחדש רק מהיום שיש בו נתון חדש והלאה – כך שסנכרון יומי שגרתי הוא פעולה על יום אחד, לא על 120.
3. עימוד מבוסס-סמן במסד. כיום העימוד בממשק קיים, אך השליפה מהמסד היא מעל תקרה קבועה (60 שורות) ולא שליפה מדורגת אמיתית. השיפור: המסד יחזיר עמוד לפי עמודת סמן (started_at > הערך האחרון שהוצג) במקום offset – כך שכל עמוד נשלף בלי לסרוק ולזרוק את כל מה שלפניו.
4. הגבלת קצב. כיום אין שום הגבלה על ה-API של המוצר עצמו. השיפור: קודם אימות/התחברות (כדי לא להגביל בטעות משתמשים אמיתיים), ואז הגבלה לפי משתמש או IP בקצב שמתאים לשימוש טבעי אך חוסם ניסיון אוטומטי לשלוח כמות גדולה של בקשות.
5. תמונות ל-Storage. כיום תמונת הפרופיל נשמרת כ-URL data בתוך שורת ה-profiles עצמה – נוח לכתוב, אבל מנפח כל שורה שנקראת בכל טעינה, גם כשהתמונה לא נחוצה. השיפור: להעביר את הקובץ ל-Supabase Storage ולשמור בשורה רק נתיב, כך שהשורה החמה שנטענת כל הזמן נשארת קלה.

6. SELECT ... FOR UPDATE SKIP LOCKED. כיום יש ריצה סדרתית יחידה, כך שאין סיכון שתי פעולות יתנגשו על אותו משתמש. בשלב שבו יהיה תור עם כמה עובדים (שיפור 1), SELECT ... FOR UPDATE SKIP LOCKED הוא המנגנון שמבטיח שרק עובד אחד ייקח משימה של משתמש מסוים בכל רגע – עובד שני שמגיע לאותה שורה מדלג הלאה במקום לחכות או לכפול את העבודה.

7. סיכומים שבועיים שמורים. כיום לוח המספרים גוזר נפח, TRIMP וקצב לכל שבוע מחדש מהריצות הגולמיות בכל טעינה – עבודה שחוזרת על עצמה בלי להשתנות. השיפור: טבלת סיכום שבועי שנכתבת פעם אחת בזמן הסנכרון (שורה לשבוע), כך שגרף של שנה שלמה נטען מ-52 שורות מוכנות במקום לחשב מאות ריצות מחדש בכל פעם.

לסיכום: המוצר מחזיק היטב לעשרות עד מאות משתמשים, ולמגבלה הראשונה שתישבר יש מספר ידוע: 200-400 משתמשים פעילים, בלולאת הסנכרון היומית. זו הסיבה שתור העבודה הוא השיפור היחיד מהרשימה שמתאים לבצע באופן יזום לפני שהכאב מגיע – נקודת השבירה שלו מחושבת, לא משוערת. שאר השיפורים שווים את העלות שלהם רק כשהשימוש בפועל יצדיק אותם.